ADAPTIVE DECISION INTELLIGENCE FOR CRISIS MANAGEMENT

A PROJECT REPORT

Submitted by

ANGELIN PRAISEE K	**2116241901004**
AKSHAYA D	**2116241901002**

in partial fulfillment for the award of
the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institute

CHENNAI

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled " **ADAPTIVE DECISION INTELLIGECE FOR CRISIS MANAGEMENT** " is the bonafide work of "**AKSHAYA D (241901002)**" who carried out the work under my supervision.

SIGNATURE

Dr. Bhuvana S, Ph.D.,

Department of Computer
Science and Engineering
(Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

Signature of the Internal Examiner

ABSTRACT

Emergency response authorities must decide, often within minutes, which incident to serve first, which resources to commit, and which route to use, while weather, roads, communication and resource availability keep changing. Static response plans and manual assessment adapt poorly to such change. This project presents Adaptive Decision Intelligence for Crisis Management, a decision-support prototype that evaluates one or more simultaneous incidents and recommends an explainable response plan. The system is implemented in Python with a Streamlit dashboard, Folium maps and Geopy geocoding, and it uses classical Artificial Intelligence techniques rather than machine learning.

A weighted multi-criteria priority engine scores each incident from 0 to 100. Candidate resources are filtered by hard constraints using constraint satisfaction, including a backtracking search with the Minimum Remaining Values heuristic. Routes are computed by A* search on a simulated road graph that accounts for blocked roads, weather and damage. A multi-objective heuristic ranks the feasible options, and a two-phase allocation procedure distributes limited, shared resources across competing incidents.

An adaptation engine detects changes such as a blocked road, unavailable resource, worsening weather or a new incident, re-runs the pipeline, compares the old and new plans and explains the reason for the change using the values actually computed. Resource bases, resources and road conditions are simulation data; only geocoding and map tile are external. Twenty automated behaviour tests, covering priority, routing, constraints, allocation and six adaptation scenarios, were executed and passed. Manual browser validation, usability evaluation and expert review remain pending. The system is intended as decision support for trained responders and does not replace them.

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

AKSHAYA D
(2116241901002)

ANGELIN PRAISEE K
(2116241901004)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	
	1.2 PROBLEM STATEMENT	
	1.3 OBJECTIVES	
	1.4 SCOPE	
	1.5 SECURITY REQUIREMENTS/ SECURITY CONSIDERATIONS	
	1.6 SDG ALIGNMENT	
2	**LITERATURE REVIEW**	6
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	
	2.2 RESEARCH GAPS	
	2.3 PROPOSED SYSTEM	
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	
3	**SYSTEM DESIGN**	14
	3.1 SYSTEM ARCHITECTURE	
	3.2 MODULE DESIGN	
	3.3 DATA FLOW/UML DIAGRAMS	
	3.4 THREAT MODEL AND	

ATTACK SURFACE
3.5 SECURITY CONTROLS

4 SYSTEM DESCRIPTION 27
4.1 DATA COLLECTION
4.2 DATA PREPROCESSING
4.2.1 ALGORITHMS / METHODOLOGY
4.2.2 RESULTS AND DISCUSSION

5 CONCLUSION 50

REFERENCES 51

APPENDIX 53

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT OF THE PROJECT	5
2.1	COMPARISON OF THE EXISTING AND THE PROPOSED SYSTEM	10
3.2	THREAT MODELLING AND SECURITY CONTROL	23
3.3	SECURITY CONTROLS AND THEIR PURPOSE	24
3.4	FUNCTIONAL REQUIREMENTS	25
3.5	NON FUNCTIONAL REQUIREMENTS	25
3.6	HARDWARE REQUIREMENTS	26
3.7	SOFTWARE REQUIREMENTS	26
4.1	INCIDENT INPUT ATTRIBUTES	27
4.2	PRIORITY FACTORS FOR THE DEMONSTRATION	30
4.3	HARD AND SOFT CONSTRAINTS	30
4.4	SCORE BREAKDOWN OF THE PRIMARY RESOURCE	33
4.5	TECHNIQUES CONSIDERED AND THEIR STATUS	35
4.6	FUNCTIONAL TEST CASES AND ACTUAL RESULTS	44

4.7	ACTUAL RESULTS OF THE SIX ADAPTATION SCENARIOS	46
4.8	ACTUAL RESULTS FOR THREE SIMULTANEOUS INCIDENTS	48
4.9	RESOURCE CONTENTION IN THE THREE-INCIDENT RUN	48
4.10	NODES EXPANDED BY A* AND DIJKSTRA	48
A.1	SAMPLE RESULTSOF THE SIX ADAPTATION SCENARIOS	57
A.2	INTERFACE TEST CASES	57

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE	15
3.2	MODULE DESIGN	16
3.3	DATA FLOW DIAGRAM – LEVEL 0	17
3.4	DATA FLOW DIAGRAM – LEVEL 1	18
3.5	UML USE CASE DIAGRAM	19
3.6	UML ACTIVITY DIAGRAM OF THE DECISION PROCESS	20
3.7	UML SEQUENCE DIAGRAM OF ADAPTIVE RE-PLANNING	21
3.8	THREAT MODEL AND ATTACK SURFACE	22
3.9	SECURITY CONTROL FLOW	24
4.1	DECISION PIPELINE	27
4.2	EFFECT OF ADAPTIVE RE-PLANNING ON THE ETA OF THE PRIMARY RESOURCE	47
4.3	NODES EXPANDED BY A* AND DIJKSTRA	49
A.1	DASHBOARD	53
A.2	DECISION PANEL	54
A.3	APP	54
A.4	MAPVIEW	55
A.5	ASTAR	55
A.6	HEURISTIC	56
A.7	LOCATION	56
D.1	INCIDENT CREATION	58
D.2	CRISIS MAP	58

D.3 DECISION INTELLIGENCE 59

D.4 RESOURCE ALLOCATION 59

D.5 ROUTE ANALYSIS 60

D.6 ADAPTIVE SIMULATION 60

D.7 SYTEM ANALYSIS 61

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

A crisis such as a flood, fire, earthquake or industrial accident forces authorities to take consequential decisions under time pressure and with incomplete information. The responders must judge how severe each incident is, how many people are affected, which resources are able to help, how quickly those resources can arrive, and what to do when several incidents compete for the same limited assets. In practice these decisions rely on predefined plans, standard operating procedures and the judgement of experienced officers. Such plans are valuable, but they are written in advance for typical situations and cannot anticipate the exact combination of weather, damaged roads, lost communication and depleted resources that occurs in a real event.

Decision Support Systems (DSS) assist decision makers by combining data, models and an interactive interface. Artificial Intelligence (AI) contributes techniques for representing knowledge, searching a space of alternatives and satisfying constraints. Classical techniques such as Constraint Satisfaction Problems (CSPs), heuristic search and multi-criteria scoring are particularly suitable for emergency planning because they are transparent: every recommendation can be traced to explicit rules, numbers and constraints. This property matters in safety-related work, where a recommendation that cannot be explained is unlikely to be trusted.

Adaptive Decision Intelligence for Crisis Management is a decision-support prototype built on these ideas. It accepts incident descriptions, places the incident on an interactive map, assesses its priority, filters the available resources by hard constraints, finds routes with A* search, ranks the feasible options with a multi-objective heuristic, allocates limited resources across competing incidents and explains the result. Its central feature is adaptation: when the situation changes, the system re-evaluates its previous decision, produces a new plan and states why the decision changed.

Delayed or inappropriate decisions lead to poor prioritisation and inefficient use of scarce resources. The growing availability of AI techniques creates an opportunity to make crisis decisions more systematic and transparent. The project was selected to explore how foundational AI methods can support faster, context-aware and explainable emergency

decision.

The project demonstrates, in a form that can be run and examined, how search, constraint satisfaction, heuristics and allocation can be combined into one decision pipeline, and how such a pipeline can respond to a changing environment. It is a decision aid for trained responders and not a substitute for them.

1.2 PROBLEM STATEMENT

Crisis situations change rapidly, which makes it difficult to assess severity, identify priorities and select resources and routes in time. Conventional approaches depend on predefined plans or manual assessment. They usually treat a disaster as a fixed type, give little help when several incidents compete for the same limited resources, and do not automatically reconsider a decision when a road is blocked, the weather deteriorates, a resource becomes unavailable or a new incident appears.

The problem addressed by this project is to design and implement a decision-support system that

- Represents incident conditions, resources, roads and constraints explicitly.
- Recommends a feasible and prioritised response plan for one or more simultaneous incidents.
- Resect the quantity and capability limits of the available resources.
- Recomputes the plan when conditions change.
- Explains each decision in terms of the values that produced it.

1.3 OBJECTIVES

The primary objective is to develop an adaptive, constraint-aware and explainable emergency-response decision-support system using classical AI techniques. The specific objectives are:

- Analyse the disaster type, severity, affected population, weather, infrastructure, communication and accessibility to compute a normalised priority score between 0 and 100.
- Apply hard constraints (availability, quantity, capability, weather limits, reachability and time window) so that impossible options are eliminated before optimisation.

- Compute routes on a road graph with A* search, taking distance, travel time, road status, weather and damage into account, and provide an alternative route.
- Rank feasible options with a transparent multi-objective score and allocate limited shared resources across several simultaneous incidents without exceeding available quantities.
- Detect changes in roads, weather, resources, incidents and accessibility, re-plan, and present the previous decision, the current decision and the reason for the change.
- Generate human-readable explanations from the actual values used by the algorithms.
- Present the situation on an interactive map and a professional dashboard, and support simulation of six demonstration scenarios.

1.4 SCOPE

The scope of the project is a working academic prototype of an emergency-response decision-support system. The functionality that is implemented is:

- Creation of one or more incidents with geocoded locations (OpenStreetMap Nominatim through Geopy, with an offline gazetteer fallback).
- Weighted multi-criteria priority engine with per-factor scores and a priority level.
- Fifteen simulated resource groups (38 units) at six simulated bases, with capabilities, capacity, mobilisation time and safe-weather limits.
- Hard-constraint filtering and a backtracking CSP with MRV and forward checking for the core assignment of resources.
- A* search on a simulated road network of 16 nodes and 25 roads with open, degraded and blocked states.
- Multi-incident allocation, contention reporting and generated explanations.
- An adaptation engine with twelve event types and six scenarios.
- A Streamlit interface with a Folium map and six tabs.

The following are outside the current scope: machine learning or deep learning models; real-time disaster, weather or traffic feeds; real road data or a live routing service; a relational or cloud database (data are stored in JSON files); user authentication; and integration with actual emergency agencies. Resource bases, resources, road conditions and the disaster-to-need profiles are simulation data and have not been validated against real operations. The quality of any recommendation depends on the accuracy of the information supplied and on the rules implemented.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Although the prototype handles simulated data, a decision-support tool for emergencies must be reliable and must not mislead its users. The following security considerations are relevant:

- Disaster type, severity, weather, infrastructure, communication and accessibility are selected from fixed lists, population is restricted to non-negative integers, and the location must resolve to coordinates before an incident is created.
- The plan must never allocate more units than exist, must never use an unavailable resource and must be recomputed from the current state after every change.
- Unsatisfiable situations, such as an unreachable incident or a missing location, must produce a clear message instead of an invalid plan.
- Incident locations are sent to the external geocoding service; no personal data are requested or stored. Operational data should be treated as sensitive in a real deployment.
- Responses from the geocoder and map-tile providers are untrusted and are used only as coordinates and images; failure of these services is handled by an offline fallback.
- User input is never executed as code or used to build file paths or queries; Python dependencies should be kept updated.
- Authentication and role-based access are not implemented and would be required for any shared deployment; the adaptation log records each event, its time and the reason for each plan change.
- Recommendations are decision support for trained responders. They are produced from simulated data and must not be treated as validated real-world emergency instructions.

1.6 SDG ALIGNMENT

The project supports the following United Nations Sustainable Development Goals (SDGs). The alignment is indicative: it describes how the project could contribute and does not claim measured impact.

Table 1.1 SDG Alignment of the Project

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 11 – Sustainable Cities and Communities	**Target 11.5:** Significantly reduce the number of deaths and the number of people affected by disasters.	Supports faster, prioritised and explainable allocation of rescue, medical and relief resources to affected populations.
2	SDG 11 – Sustainable Cities and Communities	**Target 11.b:** Increase the number of cities adopting integrated policies and plans towards resilience to disasters.	Demonstrates a computational approach to disaster-response planning that can complement preparedness plans.
3	SDG 13 – Climate Action	**Target 13.1:** Strengthen resilience and adaptive capacity to climate-related hazards and natural disasters.	Models adaptation to changing weather, road and resource conditions during hydro-meteorological events such as floods.
4	SDG 9 – Industry, Innovation and Infrastructure	**Target 9.5:** Enhance scientific research and upgrade technological capabilities.	Applies classical AI search, constraint satisfaction and heuristic optimisation to a practical public-safety problem.

CHAPTER 2

LITERATURE REVIEW

Literature Survey:

Decision support systems play an important role in emergency management by helping decision-makers assess crisis situations, allocate resources, and develop effective response strategies. Turoff et al. [9] proposed the Dynamic Emergency Response Management Information System (DERMIS), which supports emergency response through information management and coordination. Their study emphasized the importance of information sharing and timely decision-making during emergencies. However, adapting response strategies to changing conditions and resource constraints remains a significant challenge.

Altay and Green [10] reviewed operations research and management science techniques used in disaster operations management. Their study highlighted the importance of systematic planning, optimization, and resource management in disaster preparedness and response. Galindo and Batta [11] further examined developments in this field and emphasized the role of analytical methods in improving emergency response decisions. These studies provide a foundation for developing intelligent systems that evaluate multiple crisis parameters before recommending appropriate actions.

Rolland et al. [16] investigated decision support approaches for disaster management, highlighting their role in evaluating alternatives and improving emergency planning. Similarly, Altay and Green [17] discussed analytical techniques for addressing resource allocation and coordination challenges in disaster operations. Their findings indicate that effective emergency management requires decision-making approaches that consider multiple operational requirements, limited resources, and competing priorities.

Mukhopadhyay et al. [15] reviewed incident prediction, detection, resource allocation, and dispatch models for emergency management. Their study highlighted the challenges of making decisions under uncertainty and rapidly changing conditions. The authors emphasized the importance of integrating prediction, resource allocation, and dispatch planning to improve emergency response. However, combining these functions into a unified system that adapts to different crisis conditions remains challenging.

Zhang et al. [20] investigated emergency response planning involving multiple resources and depots while considering the possibility of secondary disasters. Their research highlighted the importance of resource availability, depot locations, and additional risks when planning emergency operations. This approach is relevant to crisis management systems that assess available resources and incident conditions before recommending suitable response strategies.

Keeney and Raiffa [7] presented methods for evaluating alternatives involving multiple objectives and trade-offs. Their work provides a theoretical foundation for multi-criteria decision-making. In crisis management, relevant criteria include incident severity, affected population, weather conditions, infrastructure status, communication availability, and resource availability. Considering these factors together can support better incident prioritization and emergency response planning.

Russell and Norvig [1] discussed artificial intelligence techniques, including search algorithms, heuristic methods, and constraint satisfaction. Hart et al. [2] introduced the A* search algorithm for finding cost-effective paths, while Dijkstra [3] developed an algorithm for determining shortest paths in graphs with non-negative edge weights. These algorithms provide a foundation for emergency route planning and identifying suitable paths to incident locations. However, practical routing must also consider road accessibility, infrastructure disruptions, and changing emergency conditions.

Mackworth [4] established important foundations for constraint satisfaction by investigating consistency in networks of relations. Haralick and Elliott [5] explored methods for improving search efficiency in constraint satisfaction problems, while Rossi et al. [6] presented comprehensive techniques for constraint programming. These approaches are relevant to emergency planning, where response decisions must satisfy constraints involving resource availability, incident severity, and operational capacity.

The United Nations Office for Disaster Risk Reduction [21] introduced the Sendai Framework for Disaster Risk Reduction 2015–2030, emphasizing disaster risk understanding, preparedness, and resilience. Similarly, FEMA [22] presented the National Incident Management System to support coordination among personnel, equipment, facilities, and communication systems during emergencies. These frameworks highlight the importance of coordinated planning and reliable information management in disaster response.

The National Institute of Standards and Technology [23] introduced the Artificial Intelligence Risk Management Framework to promote trustworthy AI systems. Its principles emphasize reliability,

transparency, accountability, and risk management. These considerations are important in crisis management applications because emergency recommendations can influence response priorities and resource allocation. Therefore, intelligent systems should provide understandable recommendations based on relevant incident information.

OpenStreetMap Contributors [24] provide geographic data that can support the visualization of roads, landmarks, and other spatial features. Nominatim [25] enables location searches and the identification of geographic coordinates using OpenStreetMap data. These resources can support location identification and route visualization in emergency response applications. However, the usefulness of route recommendations depends on the accuracy of geographic information and the availability of current road conditions.

Based on the reviewed literature, existing research has contributed significantly to disaster management, decision support, resource allocation, artificial intelligence, and route planning. However, integrating these capabilities into a unified framework that adapts to changing crisis conditions remains an important challenge.

The proposed Adaptive Decision Intelligence for Crisis Management addresses this challenge by combining crisis assessment, intelligent decision-making, resource-aware planning, and location-based visualization. The system considers disaster type, severity, affected population, weather conditions, communication availability, infrastructure status, and available resources to recommend suitable response strategies. It also supports incident location identification and potential route visualization. By integrating these capabilities, the proposed system aims to improve the adaptability and efficiency of emergency response planning.

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Emergency response has traditionally relied on manual monitoring, predefined emergency plans, standard operating procedures and command structures in which officers assess the situation and assign resources. These arrangements provide discipline and clear responsibility, but the plan is fixed in advance, resource assignment is largely manual, and re-planning after a change depends on human effort and time.

Geographic Information Systems (GIS) and the Global Positioning System (GPS) are widely used for disaster mapping and risk assessment; remote sensing and satellite data support damage assessment; Internet of Things (IoT) sensors support monitoring; and rule-based alert systems issue warnings. These technologies provide situational awareness, but they are mostly monitoring-focused: they show what is happening and where, and leave the choice of response to the human decision maker.

Turoff et al. described the design principles of a Dynamic Emergency Response Management Information System (DERMIS), emphasising that such systems must support changing roles, information and decisions during an event [9]. Surveys of operations research in disaster operations management by Altay and Green [10] and by Galindo and Batta [11] show that optimisation and planning models have been applied to facility location, relief distribution, evacuation and resource allocation, and they also point out that many models assume static conditions and that more work is needed on dynamic, uncertain environments. Decision Support Systems in general combine data, models and an interface to assist, rather than replace, decision makers [8].

Operational emergency information systems typically rely on authentication and role-based access control, encrypted communication, secure storage, audit logging and redundancy so that the system remains available during an event. Their effectiveness depends on the deployment and is outside the scope of this prototype, which implements input validation, safe error handling and an event log, and lists the other controls as future work (Section 3.5).

Table 2.1 Comparison of the Existing Approach and the Proposed System

Aspect	Existing / conventional approach	Proposed system
Response plan	Predefined, written in advance	Generated from the current incident conditions and resource state
Resource choice	Manual judgement; compatible resources are often listed together	Candidates are filtered by hard constraints and ranked by a transparent multi-objective score
Competing incidents	Handled by officers; no systematic quantity check	Incidents are ranked and shared resources are allocated without exceeding available units
Routing	Static maps or manual route choice	A* routing with blocked-road avoidance and a next-best route
Change handling	Manual re-assessment	Adaptation engine re-plans and explains the change
Explanation	Rationale mainly in the officer's judgement	Explanations generated from the values used by the algorithms
Data	Real operational data	Simulation data (resources, roads) plus external geocoding; no live feeds

2.2 RESEARCH GAPS

The review of conventional practice and of the literature identifies the following gaps that the project addresses within the limits of a prototype:

- Many technologies visualise hazards but do not recommend which resource to send, from where and by which route.
- Fixed plans do not automatically respond to blocked roads, weather changes, resource loss or new incidents.
- Listing every compatible resource ignores quantity, distance, route feasibility, scarcity and competing incidents.
- Impossible options are not systematically eliminated before ranking.
- Allocation of shared resources across incidents is difficult to do consistently by hand.
- Recommendations often lack a trace of the factors that produced them.
- Decision tools need input validation, safe failure behaviour and clear separation of simulated and real data.

The project does not claim to be completely novel or superior to operational systems; it is a prototype that combines established techniques into an explainable and adaptive pipeline.

2.3 PROPOSED SYSTEM

The proposed system is an AI-based adaptive decision-support system for emergency response planning. It treats each decision as a sequence of questions, each answered by a recognised technique: how urgent is the incident (weighted priority scoring); which solutions are feasible (constraint satisfaction); which feasible solution is better (multi-objective heuristic); which route is better (A*); how limited resources should be distributed (allocation); what must change when the environment changes (adaptation engine); and why the system decided as it did (explainability).

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The system works as follows:

- The operator enters the disaster type, severity, affected population, location, weather, infrastructure, communication and accessibility. The location is geocoded and the map centres on it.
- A normalised score from 0 to 100 and a level (CRITICAL, HIGH, MODERATE, LOW) are computed.
- The needs of the disaster type (for example water rescue and medical care for a flood) are derived, and every resource group with a relevant capability becomes a candidate for each need.
- Candidates that violate a hard constraint are rejected, with the reason recorded.
- A* computes the route and estimated time from each candidate's base to the incident.
- easible candidates are scored and units are allocated, core needs first, without exceeding available quantity.
- The dashboard shows the recommended action, the route, the resource classification and the reasons.
- When the operator or the simulation changes the conditions, the pipeline runs again and the previous and current decisions are compared.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- Analyse crisis information and assess the severity and impact of each incident.
- Prioritise incidents and response actions based on current conditions.
- Generate context-aware, explainable resource and route recommendations.
- Respect resource quantity, capability, location and route constraints.
- Allocate shared resources fairly and efficiently across simultaneous incidents.
- Adapt recommendations when roads, weather, resources or incidents change.
- Present the result through an interactive map-based dashboard and to demonstrate it through simulated scenarios.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

- Impossible options are removed before optimisation, so the plan never uses unavailable resources or exceeds available units.
- Resources receive different scores and unit counts according to suitability, distance, risk and scarcity, instead of every compatible resource receiving one unit.
- Routes are computed on a graph and recomputed when a road is blocked.
- Incidents are ranked and compete for shared resources under explicit rules, and contention is reported.
- A change in conditions leads to a new plan and adapt before/after plan.

CHAPTER 3

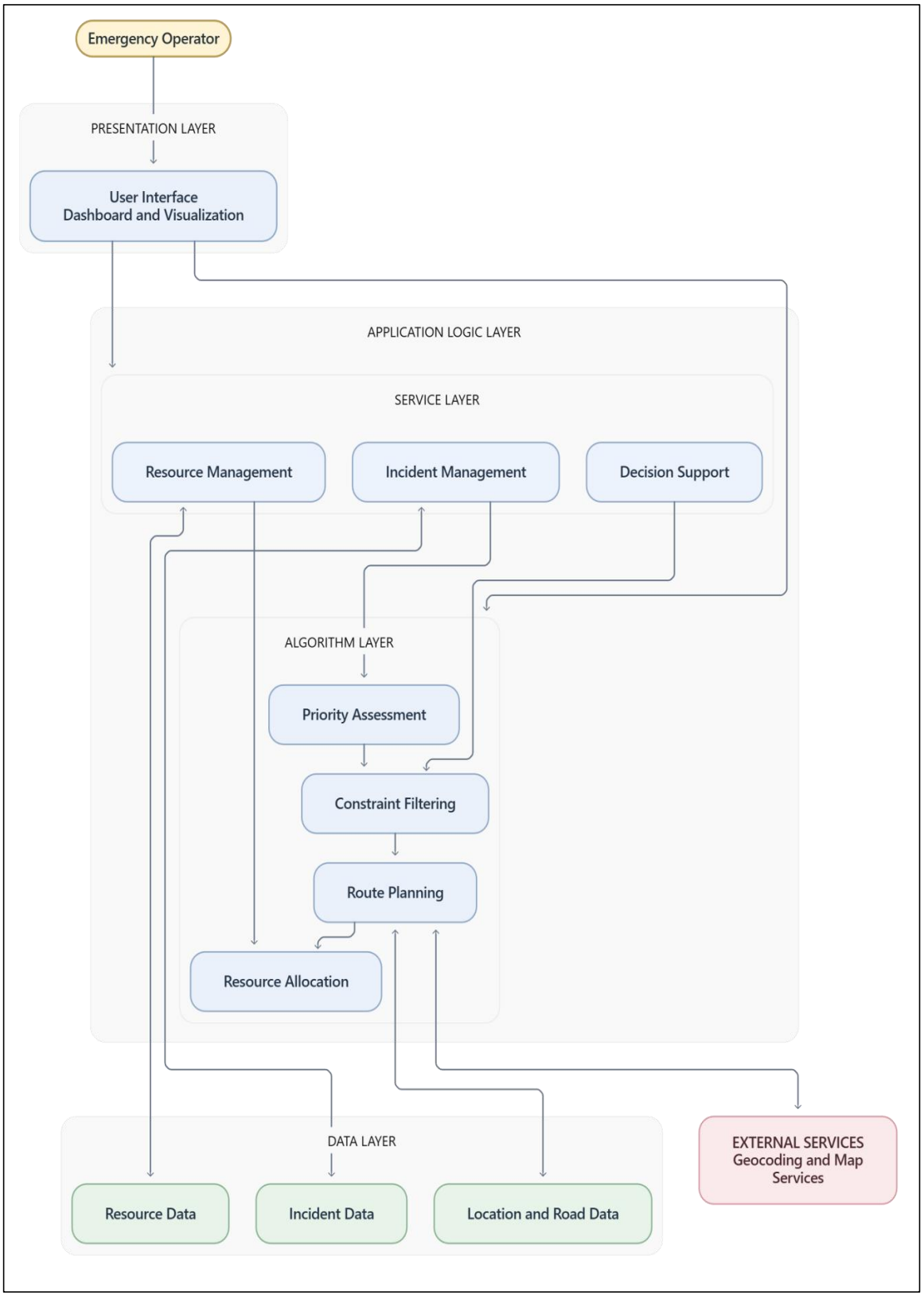
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system follows a layered architecture in which dependencies flow downwards only: the presentation layer calls services, services call algorithms, and algorithms operate on plain data models. Algorithms never import Streamlit, so each can be explained and tested independently. Figure 3.1 shows the layers and the external services.

The presentation layer (Streamlit) collects inputs and displays the map, plan and explanations. The service layer manages incidents, resources, locations and routes and runs the decision and adaptation engines. The algorithm layer contains the priority, constraint, heuristic, A* and allocation procedures. The data layer consists of JSON files. OpenStreetMap Nominatim (through Geopy) and map tiles are the only external services; everything else, including resource bases, resources and road conditions, is simulation data.

Figure 3.1 System Architecture



3.2 MODULE DESIGN

The project is organised into the modules shown in Figure 3.2.

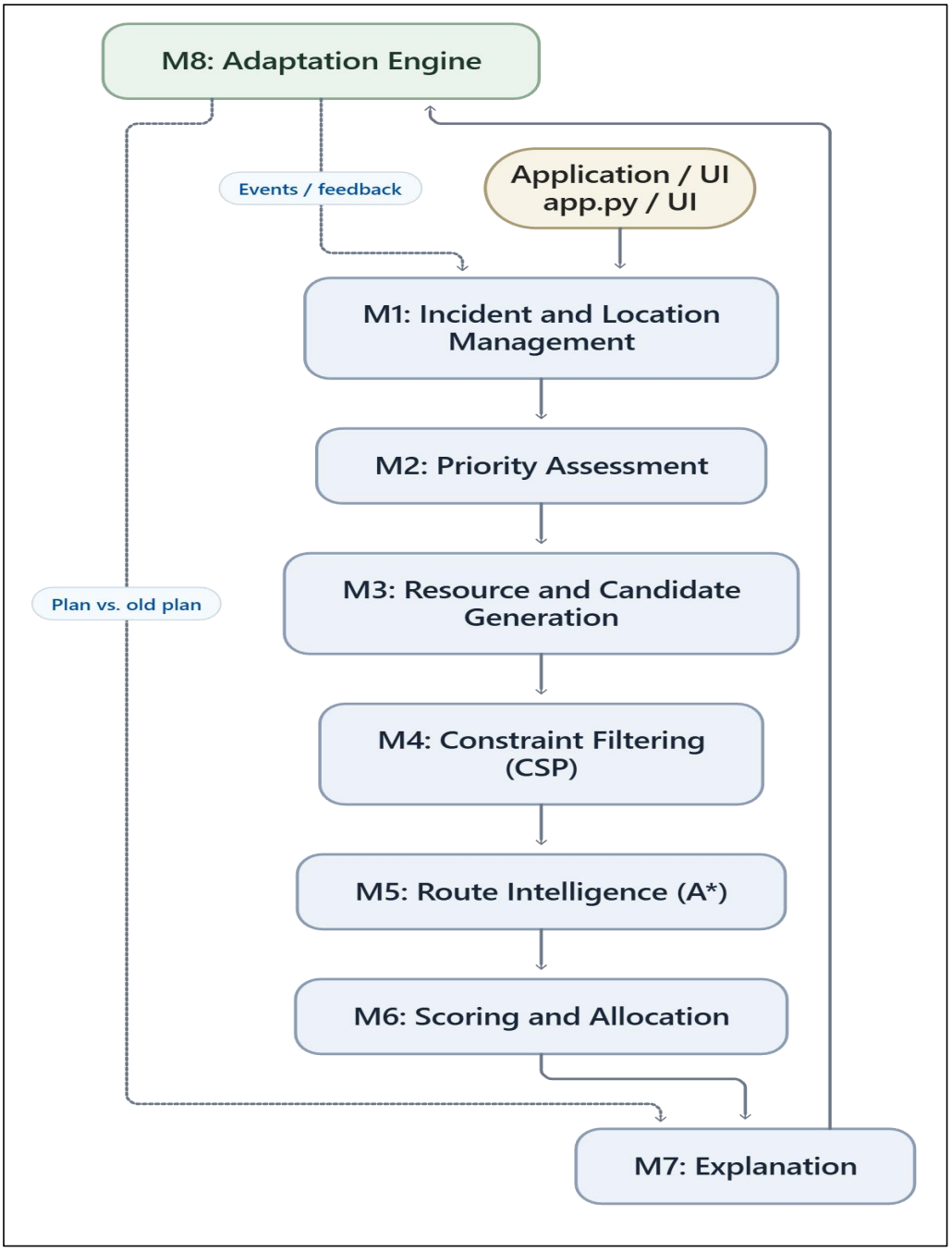


Figure 3.2 Module Design

3.3 DATA FLOW / UML DIAGRAMS

Data flow diagrams show how information moves through the system and UML diagrams show how the operator interacts with it.

3.3.1 Data Flow Diagram (DFD – Level 0)

At the highest level the operator supplies incident details and simulated events, and the system returns the priority, response plan, routes and explanations. The only external entity besides the operator is the geocoding service.The data flow of the system is depicted in Figure 3.3.

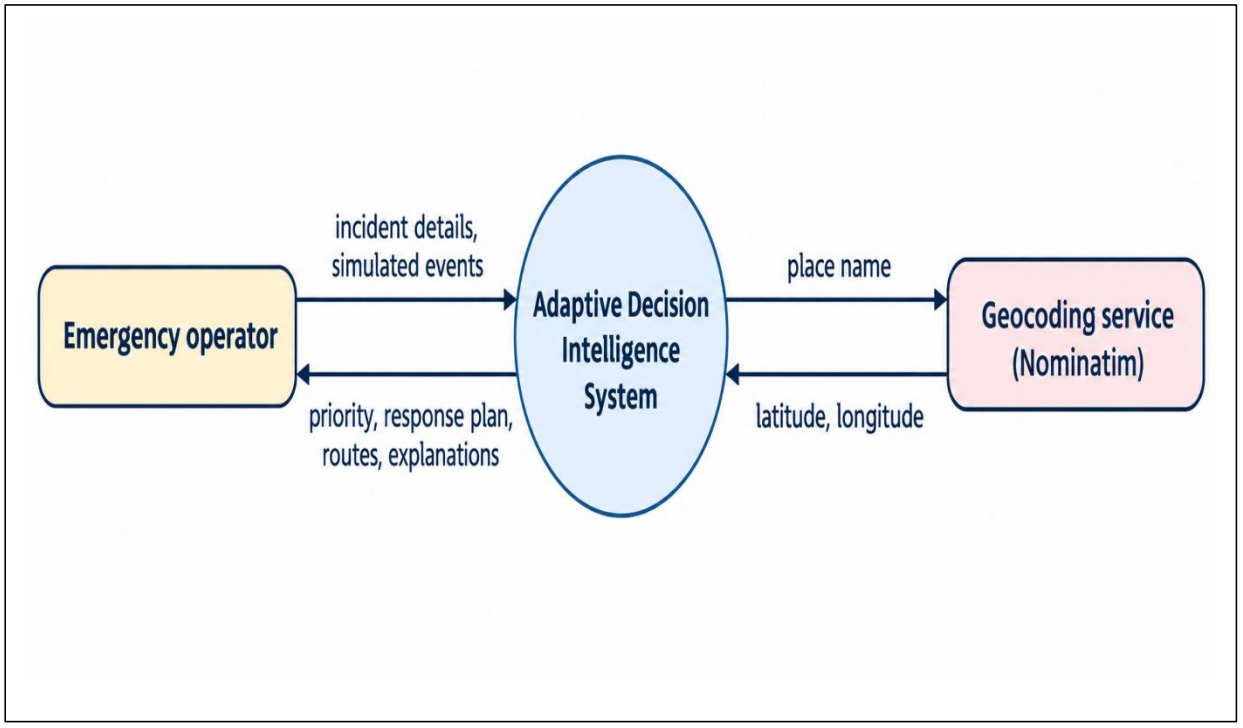


Figure 3.3 Data Flow Diagram – Level 0

3.3.2 Data Flow Diagram (DFD – Level 1)

Figure 3.4 expands the system into seven processes and four data stores. Process 7.0 modifies the stores and triggers the earlier processes again, which is the adaptive loop.

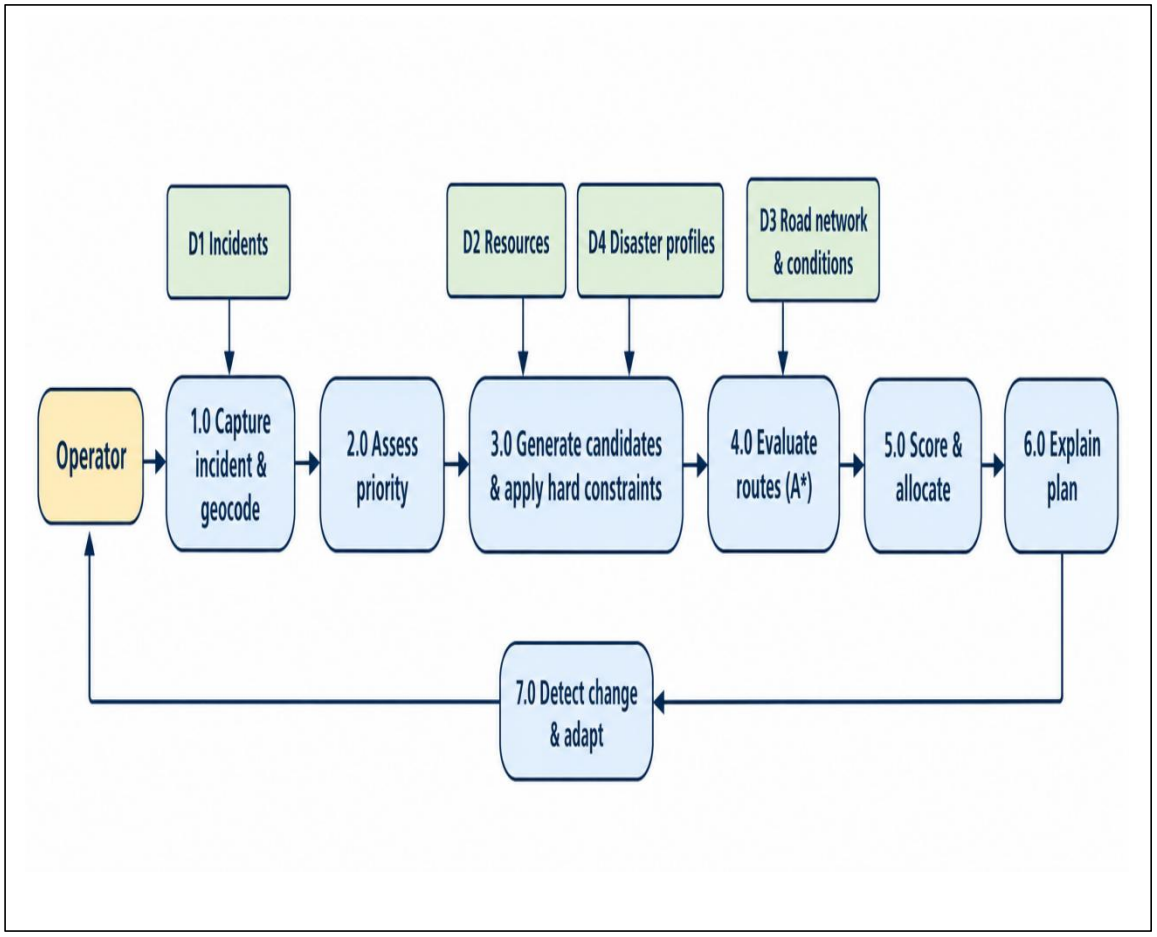


Figure 3.4 Data Flow Diagram – Level 1

3.3.3 UML Use Case Diagram

Figure 3.5 below is the dashboard supports for the emergency operator.

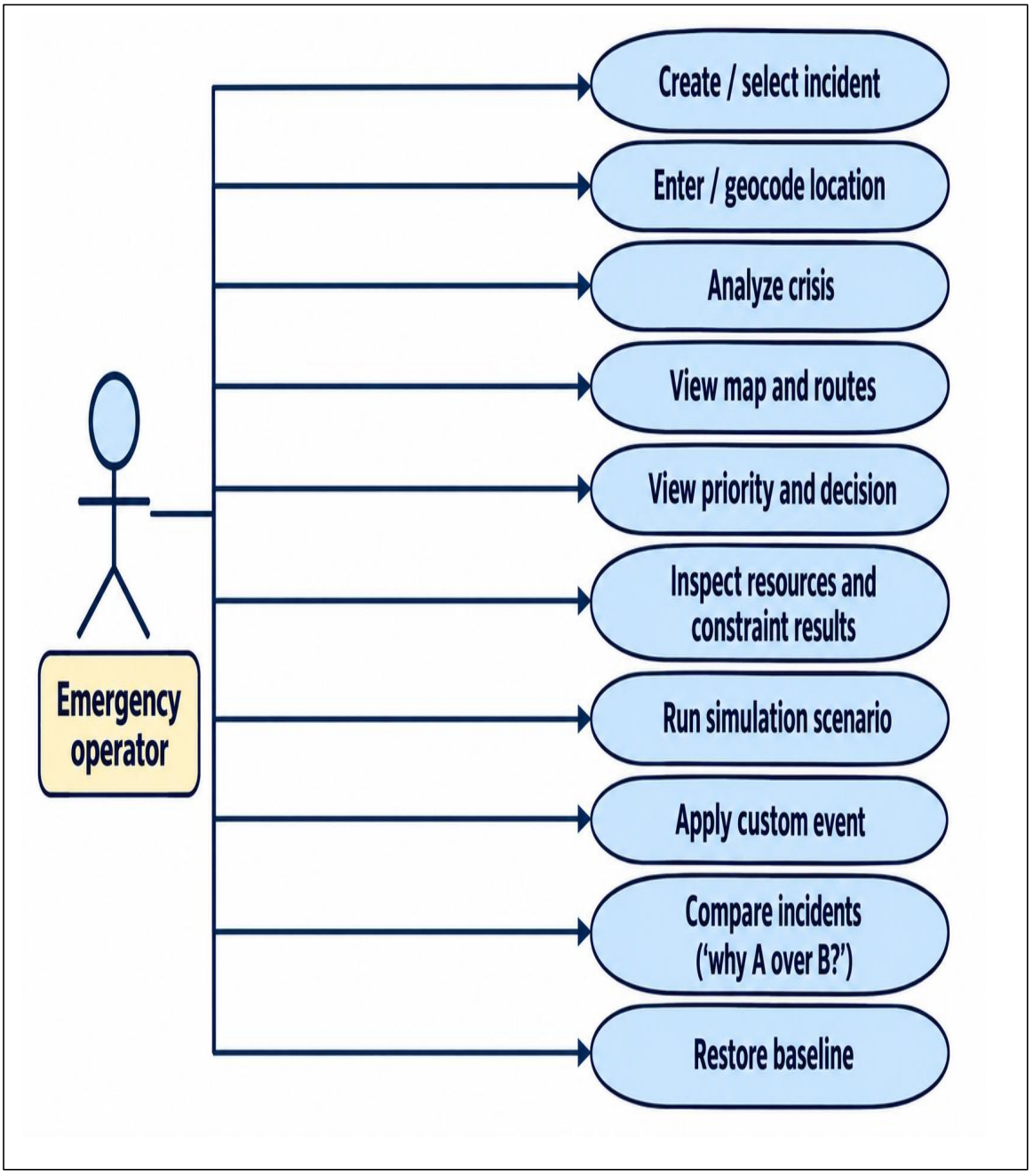


Figure 3.5 UML UseCase Diagram

3.3.4 UML Activity Diagram

Figure 3.6 shows the decision-making process from incident entry to plan display and the loop that is entered when conditions change.

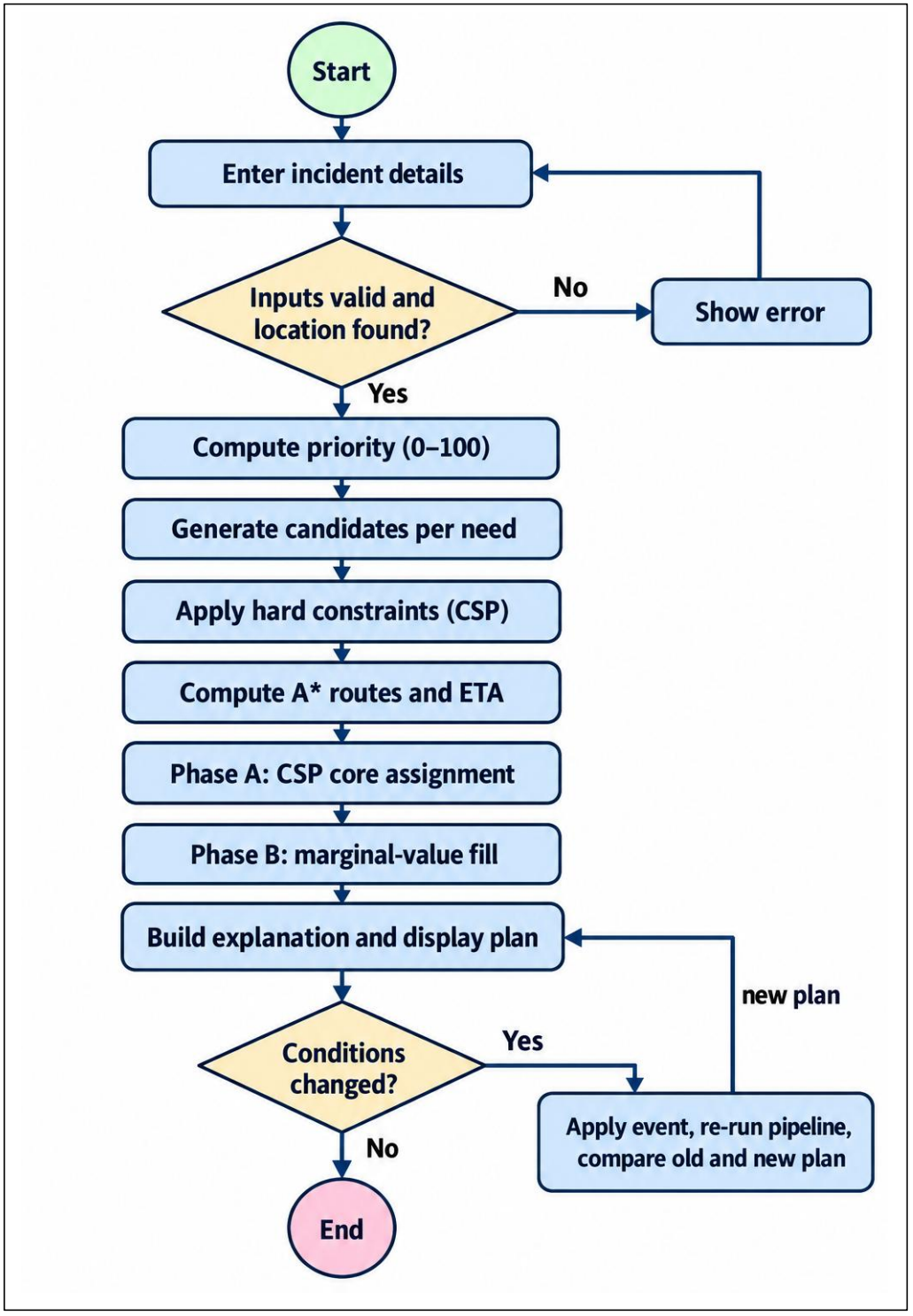


Figure 3.6 UML Activity Diagram of the Decision Process

3.3.5 UML Sequence Diagram

Figure 3.7 shows the interaction when an event such as a road blockage is applied. The adaptation engine updates the world state, asks the decision engine for a new plan and compares it with the previous plan.

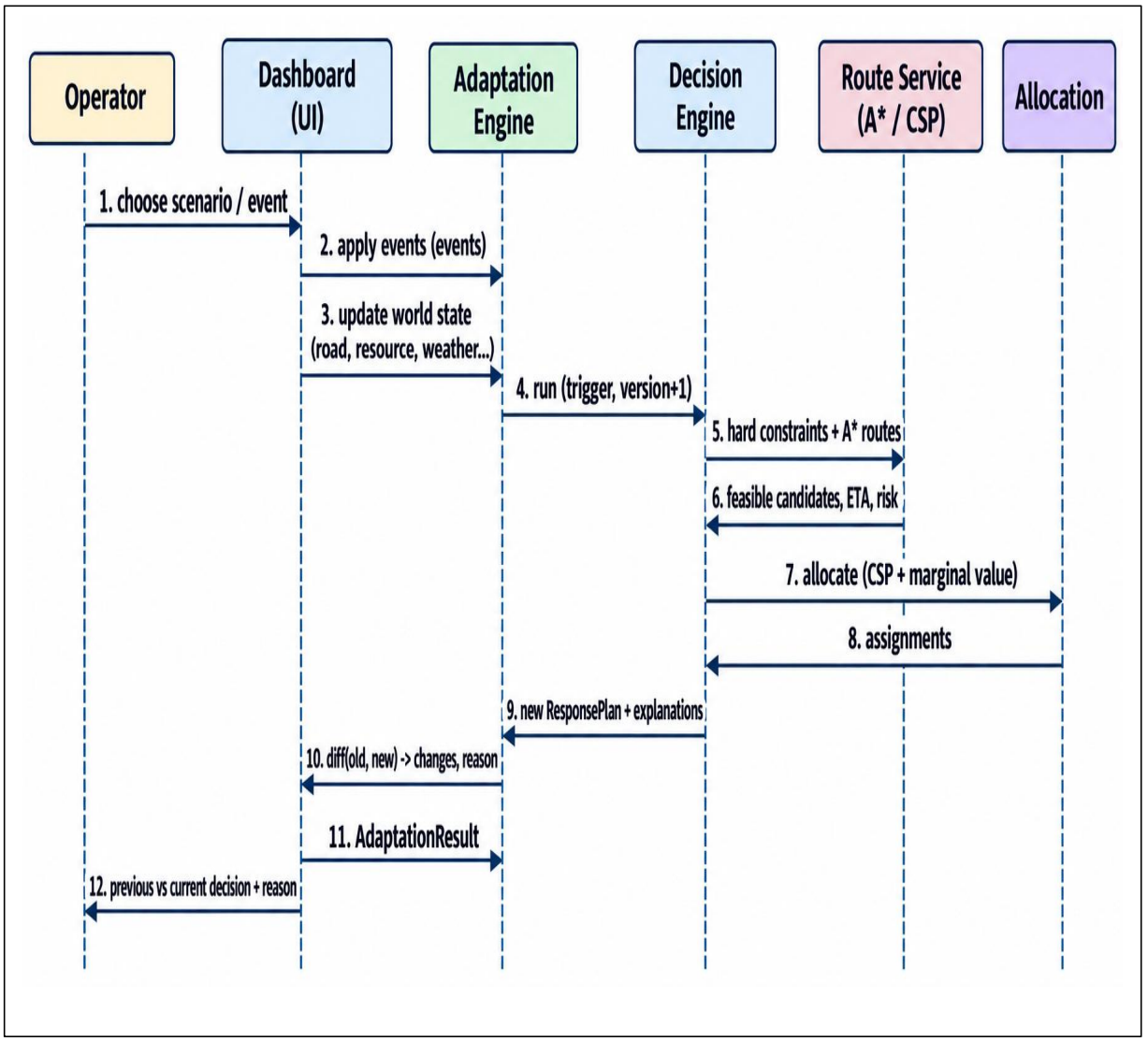


Figure 3.7 UML Sequence Diagram of Adaptive Re-planning

3.4 THREAT MODEL AND ATTACK SURFACE

The attack surface of the prototype consists of the user-input fields, the Streamlit web interface, the Python application logic, the JSON data files and the calls to external services (geocoder and map tiles). Figure 3.8 shows the threats and where the controls act; Table 3.2 lists them.

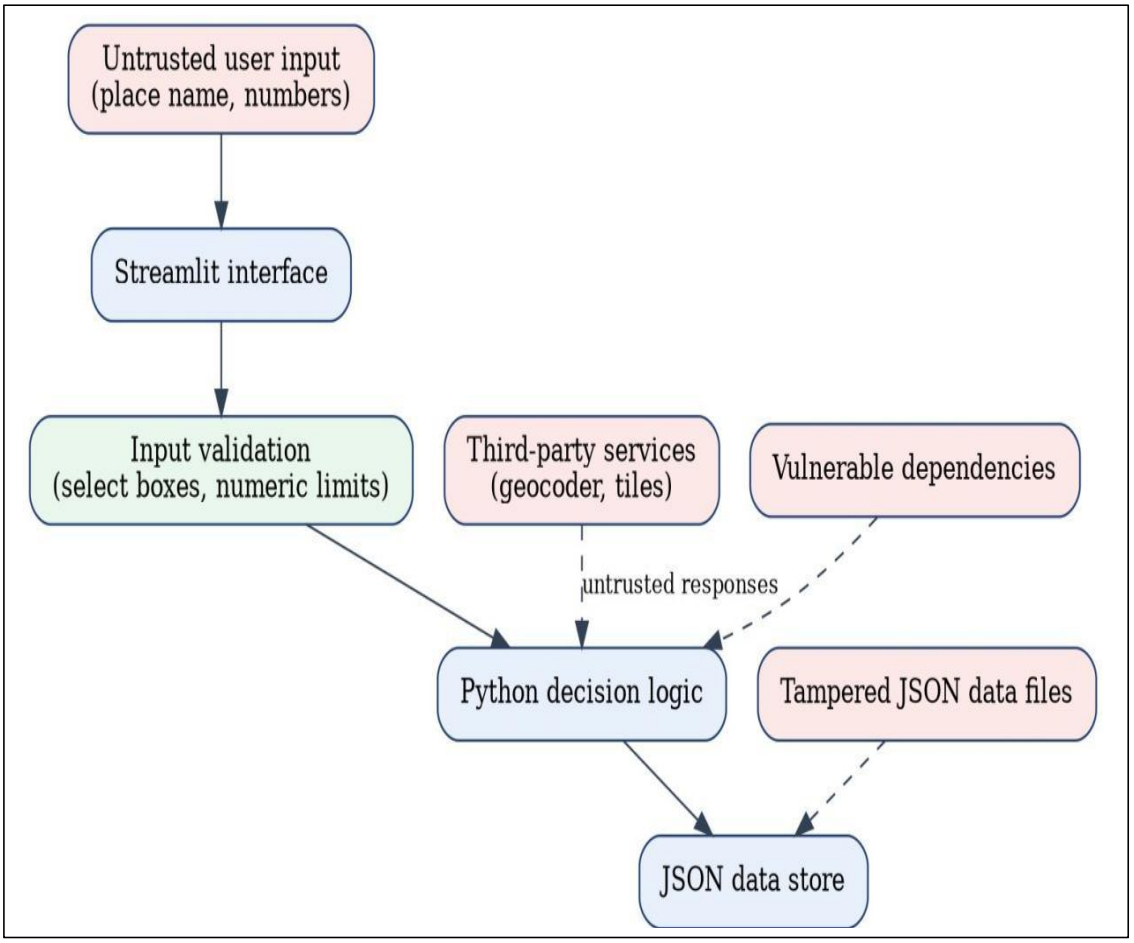


Figure 3.8 Threat Model and Attack Surface

Table 3.2 Threat Modelling and Security Control

Threat	Attack surface	Possible impact	Controls
Invalid or malicious input	Form fields, location text	Wrong or failed analysis; injection attempts	Enumerated lists and numeric limits; location must geocode; input never executed
Tampered data files	JSON data store	Wrong resources, roads or needs	File permissions, check sums or database with access control
Untrusted external responses	Geocoder, map tiles	Wrong coordinates; unavailable service	Use only coordinates; offline fallback; label data source
Unauthorised access	Web interface	Disclosure or alteration of operational data	Authentication, roles, HTTPS
Vulnerable dependencies	Python packages	Compromise of the host	Pin and update dependencies
Misleading output	Decision logic	Unsafe reliance on simulated recommendations	Explanations, simulation labels, decision-support disclaimer

3.5 SECURITY CONTROLS

Security controls protect the application, preserve data integrity and keep operation reliable. Figure 3.9 shows the control flow applied to every request, and Table 3.3 summarises the controls.

Table 3.3 Security Controls and Their Purpose

Security control	Purpose
Input validation	Reject invalid types, ranges and unknown locations
Constraint validation	Never produce a plan that uses unavailable resources or exceeds units
Safe error handling	Show clear messages without internal details; report infeasible situations
Data-source labelling	Distinguish real (external) from simulated data
Audit log	Record events, times and reasons for plan changes

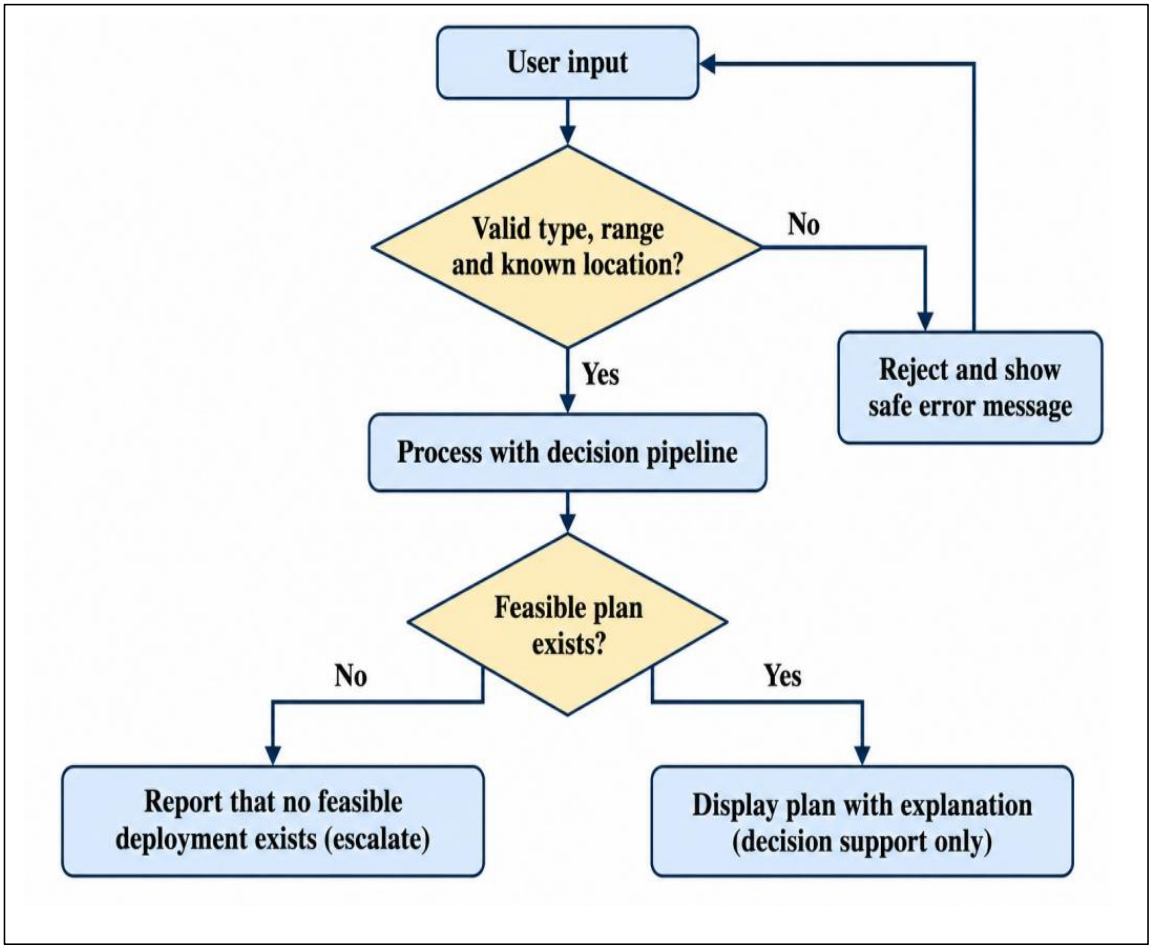


Figure 3.9 Security Control Flow

3.6 SYSTEM REQUIREMENTS

This section records the requirements that the implementation satisfies. It extends the template headings of this chapter so that the requirements are documented together with the design.

Table 3.4 Functional Requirements

ID	Requirement	Satisfied by
FR1	Create incidents with type, severity, population, location, weather, infrastructure, communication and accessibility	ui/dashboard.py, incident_manager.py
FR2	Geocode the location and centre the map on it	location_service.py, map_view.py
FR3	Compute priority (0–100), per-factor scores and level	priority.py
FR4	Reject infeasible resources and record the reasons	csp.py
FR5	Compute routes with A*, avoid blocked roads, provide an alternative	astar.py, route_service.py
FR6	Rank feasible resources with a transparent score and allocate units within available quantity	heuristic.py, allocation.py
FR7	Support multiple simultaneous incidents and explain contention	decision_engine.py, explainer.py
FR8	Re-plan on changes and show previous decision, current decision and reason	adaptation_engine.py, simulation_panel.py
FR9	Provide six simulation scenarios and custom events	scenarios.py, simulation_panel.py
FR10	Present map, decision, resource, route, simulation and analytics views	ui/*

Table 3.5 Non-Functional Requirements

ID	Requirement	How addressed
NFR1	Explainability: decisions traceable to values	Score breakdowns and generated explanations
NFR2	Modularity: UI, services, algorithms, data separated	Layered package structure
NFR3	Maintainability and testability	20 automated tests; algorithms

		independent of the UI
NFR4	Usability: dashboard with tabs and map	Streamlit interface (usability not yet evaluated)
NFR5	Reliability: no invalid plans	Hard constraints and tested invariants
NFR6	Portability: no paid services	Open-source libraries; offline fallback
NFR7	Performance: responsive for small networks	Plan computation for three incidents completes quickly on a development machine (not formally benchmarked)

Table 3.6 Hardware Requirements

Component	Minimum specification
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent
Memory	8 GB RAM
Storage	256 GB SSD (the application itself needs little space)
Display	1366×768 or higher (wide layout)
Network	Internet connection for geocoding and map tiles; optional with the offline gazetteer (tiles then unavailable)

Table 3.7 Software Requirements

Software	Purpose
Python 3.10 or later (3.12 used for the tests reported here)	Programming language
Streamlit	Web dashboard [14]
Folium and streamlit-folium	Interactive maps [15]
Geopy	Geocoding client for Nominatim [16], [17]
pandas	Tables and charts in the dashboard
Visual Studio Code (or any editor); a modern web browser	Development and use

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The system does not use a training dataset. Its data come from three sources, which the dashboard and the report keep distinct.

4.1.1 Operator input:

The operator supplies the attributes in Table 4.1 through the incident form or through the simulation controls.

Table 4.1 Incident Input Attributes

Attribute	Type / allowed values	Use in the system
Disaster type	Flood, Fire, Earthquake, Cyclone, Landslide, Industrial Accident	Selects the needs profile
Severity	Low, Moderate, High, Critical	Priority; fraction of people at risk; time window
Population affected	Non-negative integer	Priority; people at risk; units required
Location	Free text, geocoded	Coordinates, map centre, route destination
Weather	Clear, Cloudy, Light Rain, Heavy Rain, Storm, Extreme Heat	Priority; travel-time penalty; safe-operation limits
Infrastructure	Intact … Severely Damaged	Priority; penalty on the access road to the incident
Communication	Normal, Limited, Unavailable	Priority
Accessibility	Easy, Moderate, Difficult, Inaccessible	Priority; access penalty; ground access blocked if Inaccessible

4.1.2 Simulation data:

Fifteen resource groups at six bases, a road network of 16 nodes and 25 roads, and need profiles for six disaster types were created for this prototype. They are not real operational

data. The base locations are placed at approximate positions in Chennai purely for demonstration.

4.1.3 External data:

Place names are geocoded with OpenStreetMap Nominatim through Geopy [16], [17], and map tiles are provided by OpenStreetMap and Esri services. If the geocoder is unavailable, a small offline gazetteer of cities and Chennai localities is used and the result is labelled as simulation data. No live weather, traffic or incident feeds are used.

4.2 DATA PREPROCESSING

- Selections come from fixed lists, population must be a non-negative integer, and the location must geocode; otherwise no incident is created.
- If the first incident is more than 60 km from the template city, the entire simulated network is translated onto the incident's city and the interface states that it was re-centred.
- Three access roads connect the incident to its three nearest network nodes. Blocked access roads, damaged infrastructure and difficult accessibility affect only these links.
- The profile of the disaster type lists needs (for example water_rescue, medical, evacuation) as core or support with a weight and a share of at-risk people. People at risk = population × fraction for the severity (0.05, 0.12, 0.25, 0.45 for Low to Critical). Units required for a need = ⌈ people ÷ (capacity × proficiency)⌉ , bounded by the need's minimum and maximum units.
- The allowed total response time is 90, 150, 240 or 480 minutes for Critical, High, Moderate and Low severity; support needs are allowed 2.5 times as long.
- Each group has a quantity, a readiness (availability) value, a mobilisation time, a capacity, proficiencies, a travel mode and unsafe weather conditions; units that are unavailable are excluded.

4.2.1 ALGORITHMS / METHODOLOGY

The decision pipeline in Figure 4.1 combines six techniques. Each one answers a specific question. Only the techniques described in this section are implemented; other techniques that were considered are listed in Table 4.5.

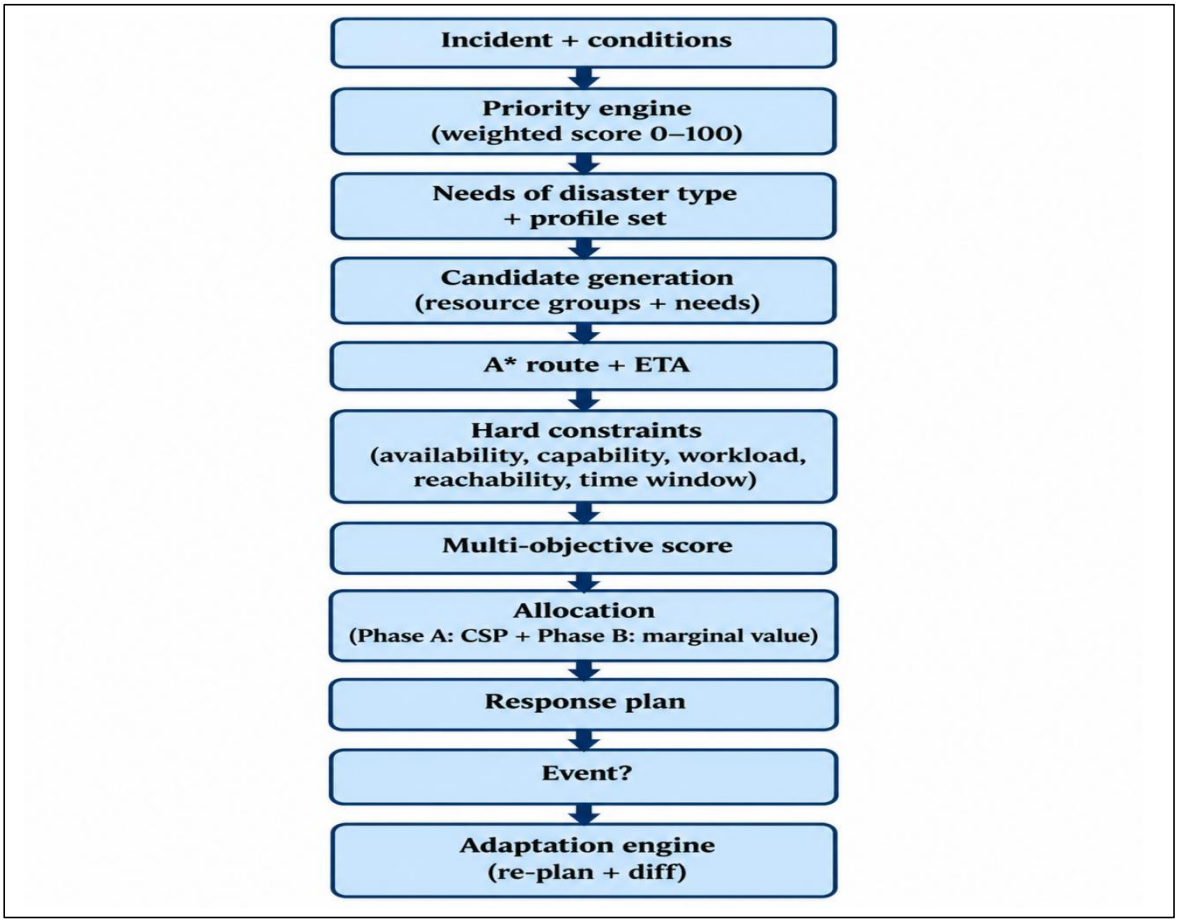


Figure 4.1 Decision Pipeline

4.2.1.1 Priority engine

The engine answers 'how urgent is the incident?'. Each factor is mapped to a score from 0 to 100 and the scores are combined with fixed weights, a weighted-sum multi-criteria model [7]: Priority = $\Sigma\, w_i \cdot s_i$, with $\Sigma\, w_i = 1$.

The weights follow the proposed design (severity 30%, population 25%, infrastructure 15%, weather 10%, communication 10%, accessibility 10%). Severity and population are consequence factors (55%), and the other four are aggravating conditions that make a response harder (45%). Population uses a logarithmic scale, $100 \cdot \log_{10}(1+P)/\log_{10}(1+10000)$, capped at 100, so that the difference between 100 and 1,000 people weighs more than the difference between 9,000 and 10,000. The levels are CRITICAL (≥80), HIGH (≥60), MODERATE (≥40) and LOW (<40).

Input and output. Input: an incident. Output: factor scores, contributions, total and level.

Table 4.2 Priority Factors for the Demonstration Flood Incident (I001)

Factor	Value	Score (0–100)	Weight	Contribution
Severity	Critical	100	30%	30.0
Population	2,500	85.0	25%	21.24
Infrastructure	Severely Damaged	100	15%	15.0
Weather	Heavy Rain	70	10%	7.0
Communication	Limited	60	10%	6.0
Accessibility	Difficult to Access	75	10%	7.5
Total			100%	86.7 (CRITICAL)

For a Critical flood affecting 2,500 people in heavy rain with severely damaged infrastructure, limited communication and difficult access, the score is $0.30\times100 + 0.25\times84.9 + 0.15\times100 + 0.10\times70 + 0.10\times60 + 0.10\times75 \approx 86.7$, a CRITICAL incident.. The scales and weights are expert-style assumptions and have not been calibrated with data; a weighted sum allows compensation between factors.

4.2.1.2 Constraint satisfaction

A CSP consists of variables, domains and constraints. Two layers are used. First, hard-constraint filtering removes every candidate that violates a hard constraint, before any optimisation. A hard constraint must hold or the solution is impossible; a soft constraint is a

preference that only affects the score. Second, a global assignment CSP chooses a resource group for each core need of each incident.

Table 4.3 Hard and Soft Constraints

Type	Constraint	Rule
Hard	Availability	Group availability > 0 and quantity > 0
Hard	Quantity	At least one unit must remain; total allocation never exceeds units (checked during allocation)
Hard	Capability	Proficiency for the need ≥ 0.25
Hard	Weather	Incident weather not in the group's unsafe-weather list (helicopter: Storm)
Hard	Reachability	A route exists (ground access fails if all routes are

		blocked or the incident is Inaccessible)
Hard	Time window	Mobilisation + travel time ≤ window of the severity (×2.5 for support needs)
Soft	Response time, route safety, availability, capacity fit, preservation of scarce resources	Scored by the heuristic (Section E)

Domains are the feasible resource groups. The constraints are: a group cannot be assigned to more variables than its units, and an incident cannot use the same group for two core needs. The solver is a backtracking search with Minimum Remaining Values variable ordering [5], forward checking through live supply counters, and value ordering by heuristic score. If no complete assignment exists, the number of unassigned variables allowed is increased from zero until a solution is found.

```
function SOLVE(variables, domains, supply):
    for skips = 0 .. |variables|:
        solution = SEARCH(variables, {}, supply, skips)
        if solution is not None: return solution, skips
function SEARCH(unassigned, assignment, supply, skips_left):
    if unassigned is empty: return assignment
    var = variable in unassigned with fewest legal values         # MRV
    for (group, score) in legal values of var ordered by score desc:   #
supply[group] > 0
        assign var = group; supply[group] -= 1                    # forward checking
        result = SEARCH(unassigned - {var}, assignment, supply, skips_left)
        undo assignment
        if result is not None: return result
    if skips_left > 0: leave var unassigned and recurse with skips_left - 1
    return None
```

If three incidents need one unit each from a group with two units and from another with one unit, the solver assigns the two best-scoring incidents to the larger group and the third to the other; if only two units exist in total, it leaves exactly one variable unassigned (tests T06). Worst case $O(d^n)$ for n variables and d values, which is exponential in general, but n is the number of core needs of the active incidents, and MRV with forward checking reduces the search.The search returns the first complete assignment found under score ordering, not necessarily the one with maximum total score.

4.2.1.3 A* route search

It expands nodes in increasing order of f(n) = g(n) + h(n), where g is the cost from the origin and h estimates the remaining cost [2]. Here the cost of a road is in minutes: length ÷ speed ×

60 × (1 + weather penalty + road penalty + 0.5 × risk + access penalty) × vehicle factor, where the weather penalty is 0 to 0.60, a degraded road adds 0.6, and the access penalty applies only to the access road of the incident. Blocked roads are removed from the graph. The heuristic is the haversine distance

[12] to the goal divided by the highest speed in the network. It is admissible and consistent because each road is at least as long as the straight line, no vehicle is faster than the fastest road, and every penalty only increases the cost; A* therefore returns an optimal path. Helicopters fly a weather-adjusted straight line.

```
function A_STAR(adj, coords, start, goal, vmax):
    h(n) = haversine(n, goal) / vmax * 60
    open = priority queue of (g(start)+h(start), start); g[start] = 0
    while open is not empty:
        u = pop node with lowest f;  if u is closed: continue;  close u
        if u == goal: return reconstruct_path(parent, goal), g[goal]
        for (v, road, cost) in adj[u]:
            if g[u] + cost < g[v]:
                g[v] = g[u] + cost; parent[v] = (u, road); push (g[v] + h(v), v)
    return NOT FOUND
```

For the demonstration flood, the boat route from the Marina base uses road R01 and the access road from Egmore (6.5 km, 21.9 min). When R01 is blocked, A* selects the direct access road from the Marina base (5.2 km, 29.5 min).O(E log V) in the worst case with a binary heap, and in practice fewer node expansions than Dijkstra's algorithm [3] (Section 4.2.2). The road network is small and simulated; the risk and weather penalties are assumptions.

4.2.1.4 Multi-objective heuristic

After filtering, the heuristic answers using a weighted sum of normalised objectives, score = $100 \cdot \Sigma\, w_i \cdot f_i$ with f_i in [0, 1] [7]. The factors are: suitability (proficiency for the need), response time (1 − ETA ÷ window), route safety (1 − route risk), availability (readiness × share of units still free), capacity fit (whether remaining units cover the people still in need) and preservation (1 − scarcity × (1 − priority ÷ 100)). The base weights are 0.38, 0.15, 0.12, 0.12, 0.08 and 0.15. The weight of response time rises from 0.15 to 0.30 and the weight of preservation falls from 0.15 to 0.05 as incident priority rises from 0 to 100, and the weights are renormalised. Urgent incidents therefore favour speed, and less urgent incidents avoid spending scarce resources. Scarcity of a resource type is the ratio of potential demand to supply, capped at 1.

Table 4.4 Score Breakdown of the Primary Resource for I001 (Rescue Boat Unit [BOAT-1])

Factor	Value (0–1)	Weight	Contribution
Suitability	0.95	0.364	34.6
Response_time	0.646	0.268	17.33
Route_safety	0.78	0.115	8.97
Availability	1.0	0.115	11.5
Capacity_fit	0.507	0.077	3.88
Preservation	0.867	0.061	5.26
Total score			81.54

Because the factors differ, the scores differ: in the demonstration flood the rescue boat (proficiency 0.95 for water rescue) is the primary resource, while medical teams are scored

for the medical need. The weights are assumptions and a different weighting could change the ranking.

4.2.1.5 Resource allocation

The earlier prototype gave every compatible resource one unit and the same score; this is replaced by two phases. Phase A uses the assignment CSP to guarantee that every core need of every incident receives at least one unit from a feasible group. Phase B fills the remaining demand greedily by marginal value, serving core needs of all incidents before any support need:

$$value = (priority \div 100) \times need\ weight \times (candidate\ score \div 100) \times unmet\ fraction$$

The unmet fraction decreases as a need is covered, so returns diminish and a nearly covered need stops outbidding an uncovered need of another incident. One unit is issued per round and the best (incident, need, group) wins; allocation stops when the best value is below 0.03, which preserves scarce resources. A group's units can never exceed its remaining quantity.

```
remaining[g] = units of g that are available
PHASE A:  assignment = SOLVE(core needs, feasible domains, remaining)       # CSP
          for each assigned (incident, need) -> give one unit
PHASE B:  for kind in (core, support):
              repeat:
                  best = argmax over incidents, needs of this kind, groups with
remaining > 0
                         of  priority/100 * need.weight * score/100 * unmet_fraction
                  if best.value < 0.03: break
                  give one unit; remaining[group] -= 1; covered += unit capacity
```

If three ambulances exist and two incidents each want two, the allocation cannot give 2 + 2; the higher-priority and better-scoring need receives the second unit and the other receives the remainder (test T11 and Table 4.9). O(R · N · C) for R allocation rounds, N needs and C candidates. Greedy allocation is not guaranteed to be globally optimal.

4.2.1.6 Adaptation engine

It supports twelve event types: road blocked, reopened or degraded; weather changed; resource unavailable or restored; resource quantity changed; new incident; severity changed; population changed; communication lost; accessibility changed; and incident resolved.

```
function APPLY_EVENTS(events):
    old_plan = current_plan
    for ev in events: update world state (network, resources or incidents)
    new_plan = DECISION_ENGINE.run(world)           # constraints, A*, scoring,
allocation again
    changes  = DIFF(old_plan, new_plan)             # priority, rank, primary, units,
route, ETA
    reason   = NARRATIVE(events, old_plan, new_plan, changes)
    current_plan = new_plan; log (events, reason)
```

The diff compares priorities, ranks, primary resources, units per group and need, route roads and ETAs. The narrative is built from these values; for a blocked road that was on an active route it states the road, the resource that used it, the previous route and ETA and the new route and ETA. If nothing material changed, the plan is confirmed as unchanged.

4.2.1.7 Explainability

Each incident plan carries a list of reasons produced by the explainer from the computed values: priority and rank, people at risk, the primary resource with its proficiency and the top-weighted score factors, units and base, route status, distance and ETA against the limit, weather penalty, competing incidents, scarcity of resources, rejected resources with the violated constraint and unmet demand. A comparison function answers why one incident received a resource instead of another using priorities, the contention table and the classification of the resource for each incident.

4.2.1.8 Techniques considered

Table 4.5 Techniques Considered and Their Status

Technique	Status	Remarks
A* search	Implemented	Routing
Dijkstra's algorithm	Used only as A* with h = 0	Benchmark for the number of expanded nodes
Constraint satisfaction, backtracking, MRV, forward checking	Implemented	Feasibility and core assignment
Weighted multi-criteria scoring and greedy allocation	Implemented	Priority, ranking, allocation
Breadth-first, depth-first, iterative deepening search	Not implemented	Uninformed search is unnecessary because edge costs are known and A* is more efficient for routing
Greedy best-first search	Not implemented	Not optimal for routing; A* preferred
Machine learning / deep learning	Not implemented	No training data; deterministic explainable logic preferred

4.2.1.9 Implementation

The system is implemented in Python with data classes for models. The hard-constraint check, for example, collects violations so that rejections can be explained:

```
def check_static(cand, incident):
    g, v = cand.group, []
    if g.availability <= 0 or g.quantity <= 0: v.append("availability: resource is
unavailable")
    if cand.cap_level < MIN_CAPABILITY:         v.append("capability: proficiency
below minimum")
    if incident.weather in g.unsafe_weather:    v.append(f"weather: {g.type} cannot
operate in {incident.weather}")
    if cand.route is None or not cand.route.found: v.append("reachability: no
feasible route")
    elif cand.eta_total > cand.window:          v.append("time_window: ETA exceeds the
limit")
    return v
```

METHODOLOGY / MODULE DESCRIPTION

MODULE 01 – INCIDENT AND LOCATION MANAGEMENT

Purpose: Collects crisis details and identifies the incident location for decision-making.

Implemented features

- Enter incident type, severity, affected population, and time.
- Select or enter the incident location.
- Validate inputs and obtain location coordinates when available.
- Store incident details for the decision pipeline.

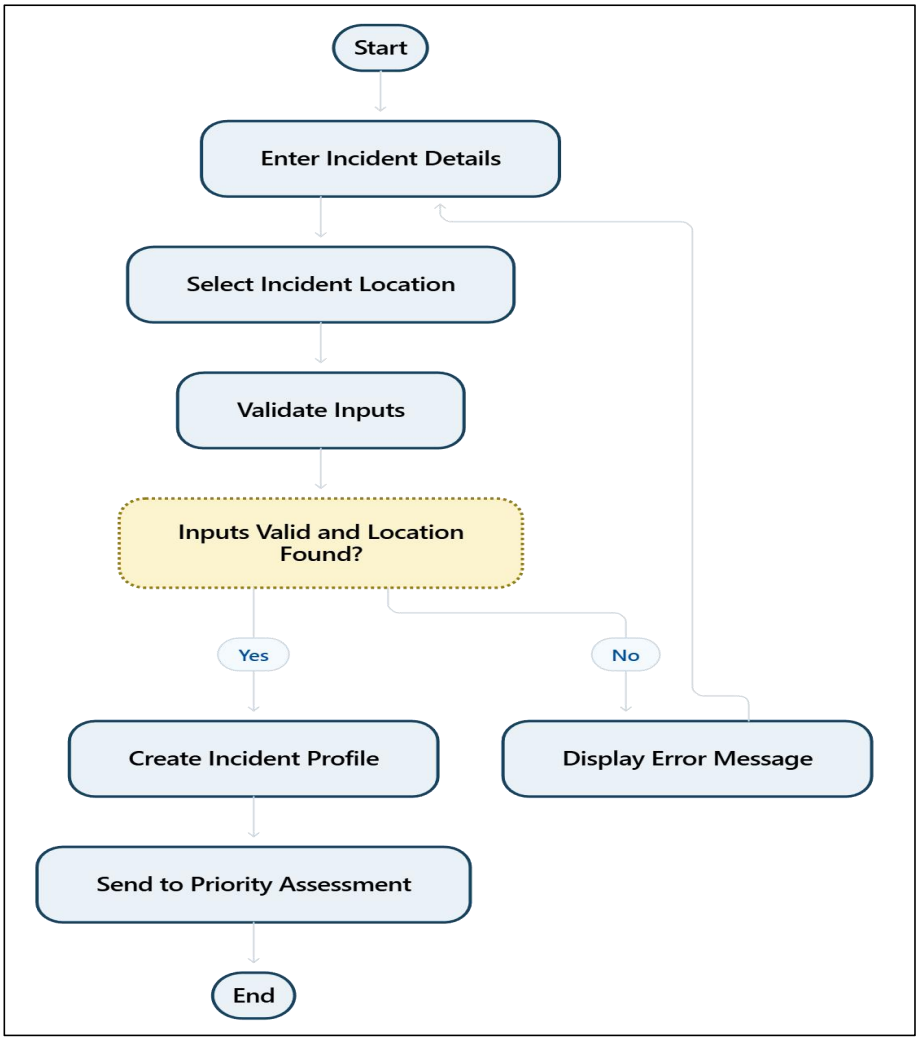


Figure 4.1 Module 01 - Incident and Location Management

MODULE 02 – PRIORITY ASSESSMENT

Purpose: Calculates an incident priority score to help determine which crisis needs attention first.

Implemented features

- Evaluate severity and population affected.
- Consider weather, infrastructure, and communication conditions.
- Calculate a weighted priority score from 0 to 100.
- Assign a priority level for decision support.

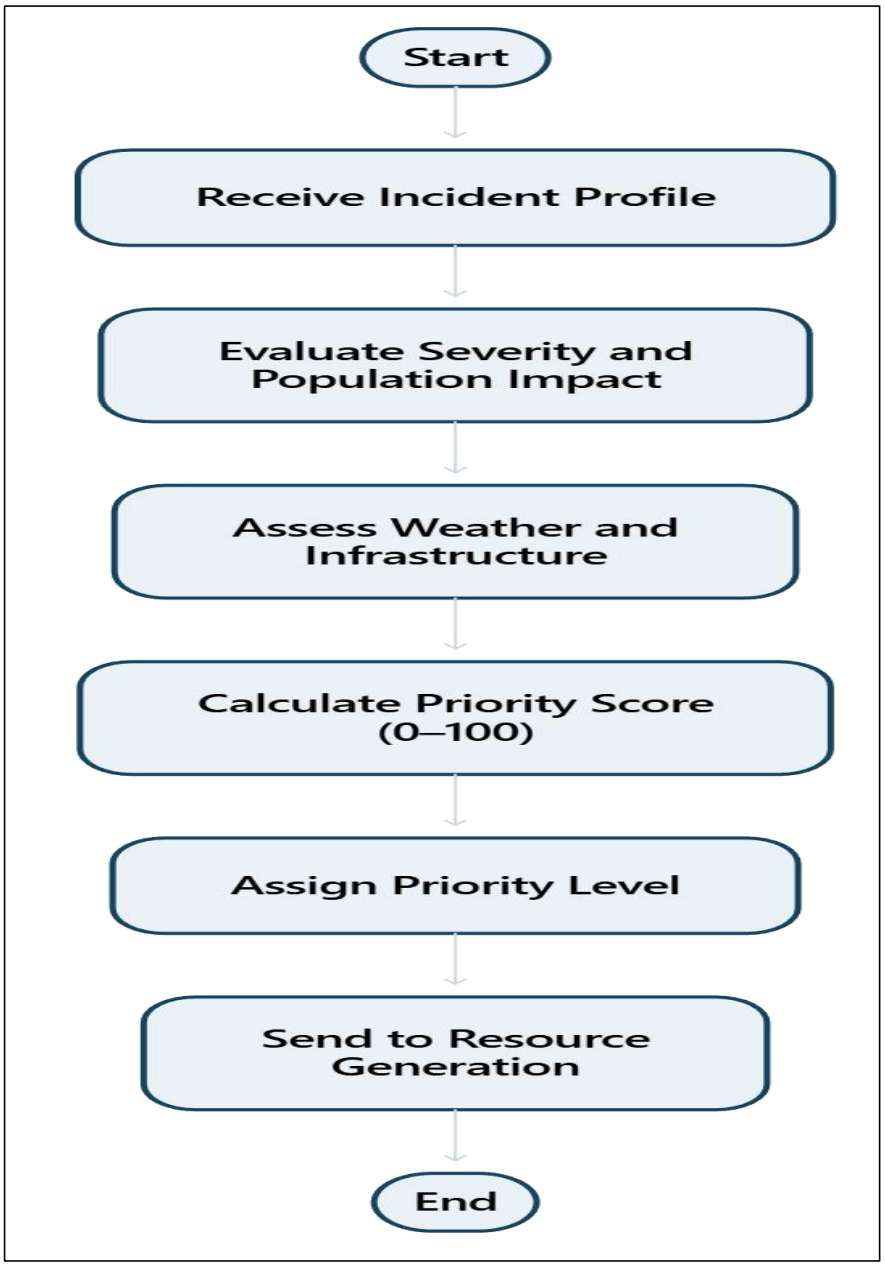


Figure 4.2 Module 02 - Priority Assessment

MODULE 03 – RESOURCE AND CANDIDATE GENERATION

Purpose: Identifies suitable response resources and creates possible assignments for the incident.

Implemented features

- Read available resources and their capabilities.
- Match resource types to incident needs.
- Generate candidate resource groups for each need.
- Exclude resources that are unavailable before detailed filtering.

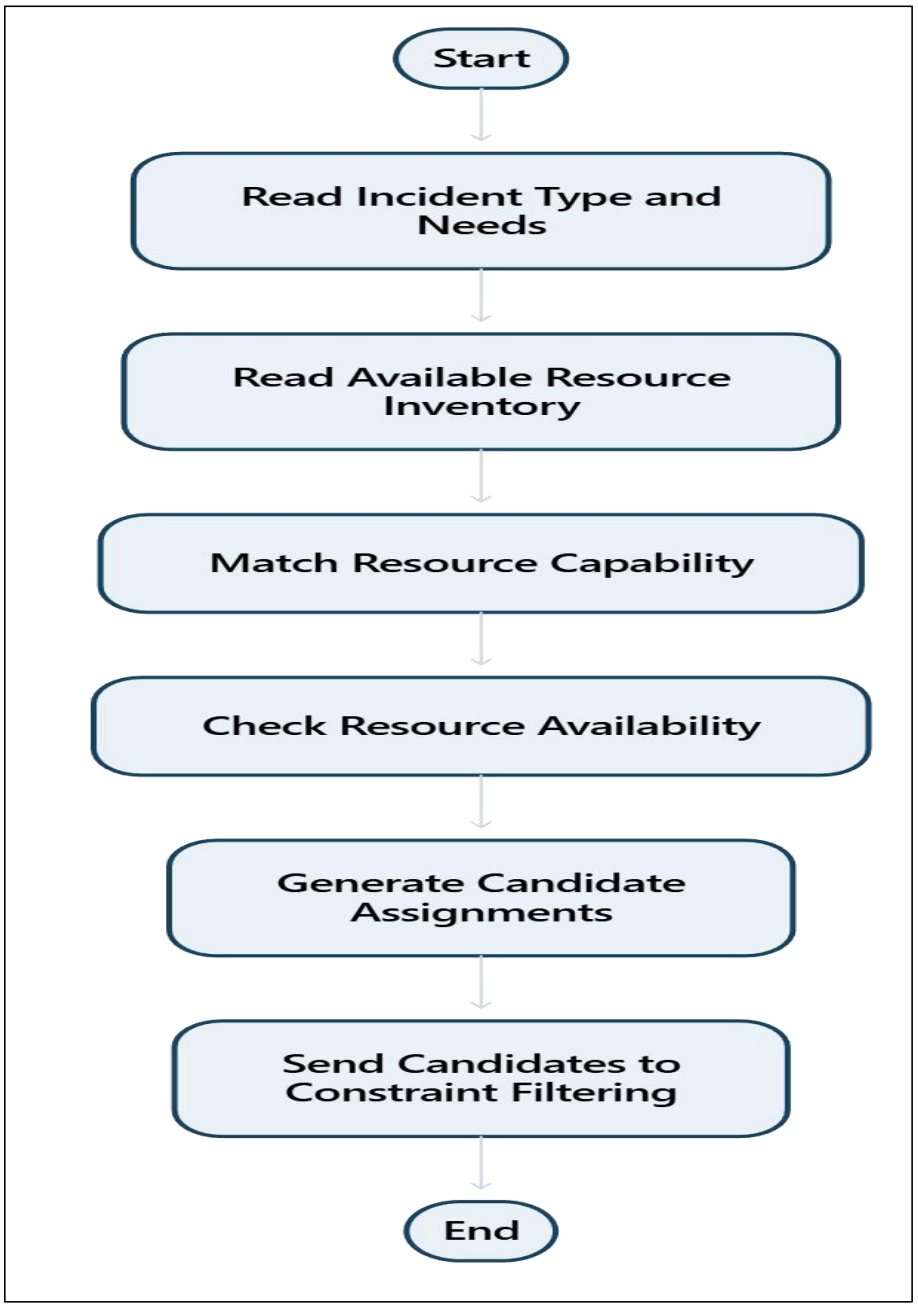


Figure 4.3 Module 03 - Resource and Candidate Generation

MODULE 04 – CONSTRAINT FILTERING (CSP)

Purpose: Removes resource assignments that violate essential operational constraints.

Implemented features

- Check resource availability and capability.
- Check workload, reachability, and time-window constraints.
- Reject infeasible combinations.
- Pass only feasible candidates to allocation.

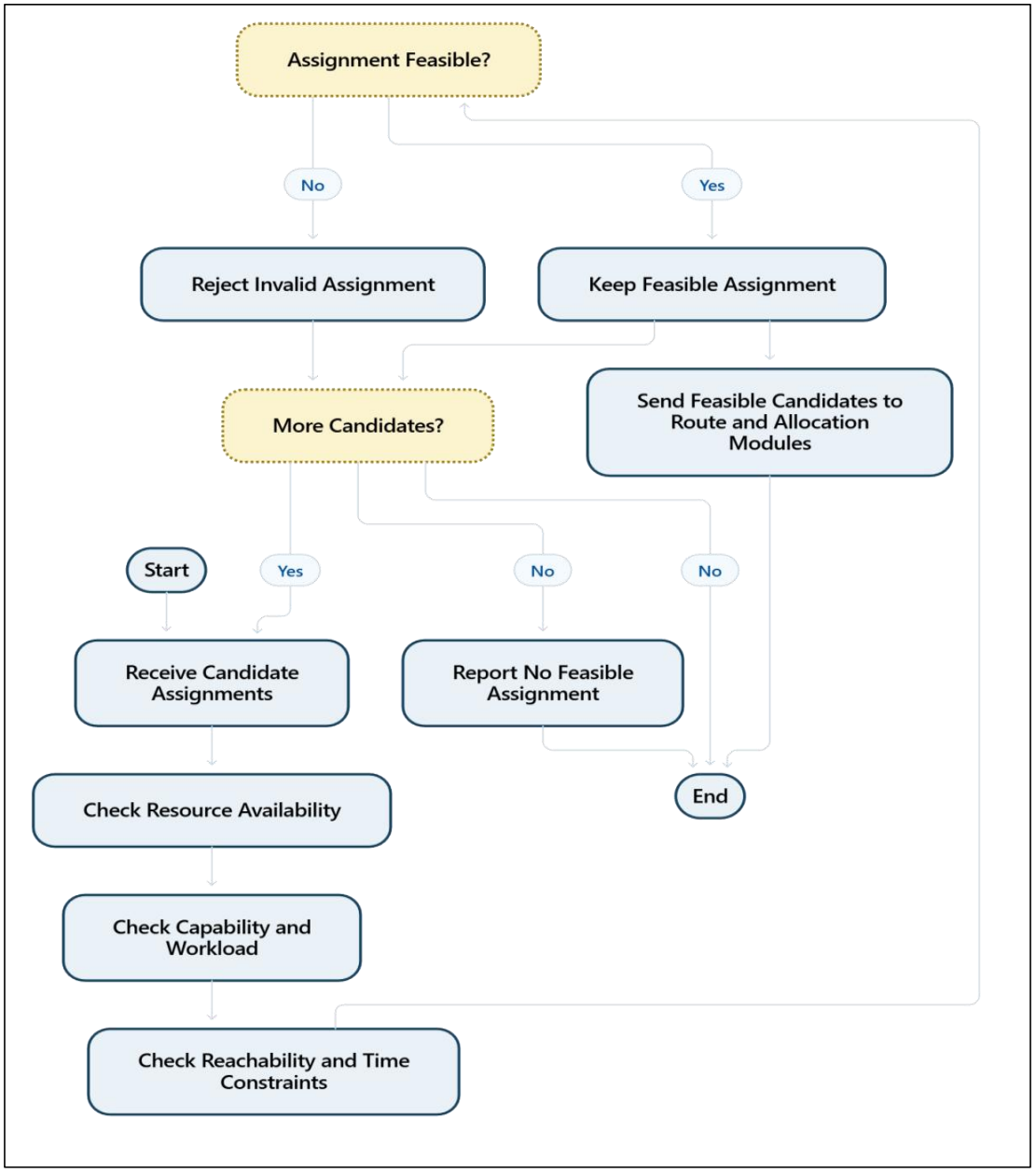


Figure 4.4 Module 04 - Constraint Filtering (csp)

MODULE 05 – ROUTE INTELLIGENCE (A*)

Purpose: Estimates routes and travel times to support safer and more practical resource deployment.

Implemented features

- Use available location and road-network data.
- Apply A* search to find a suitable route when graph data is available.
- Estimate travel distance and time.
- Mark routes as unavailable when a valid path cannot be found.

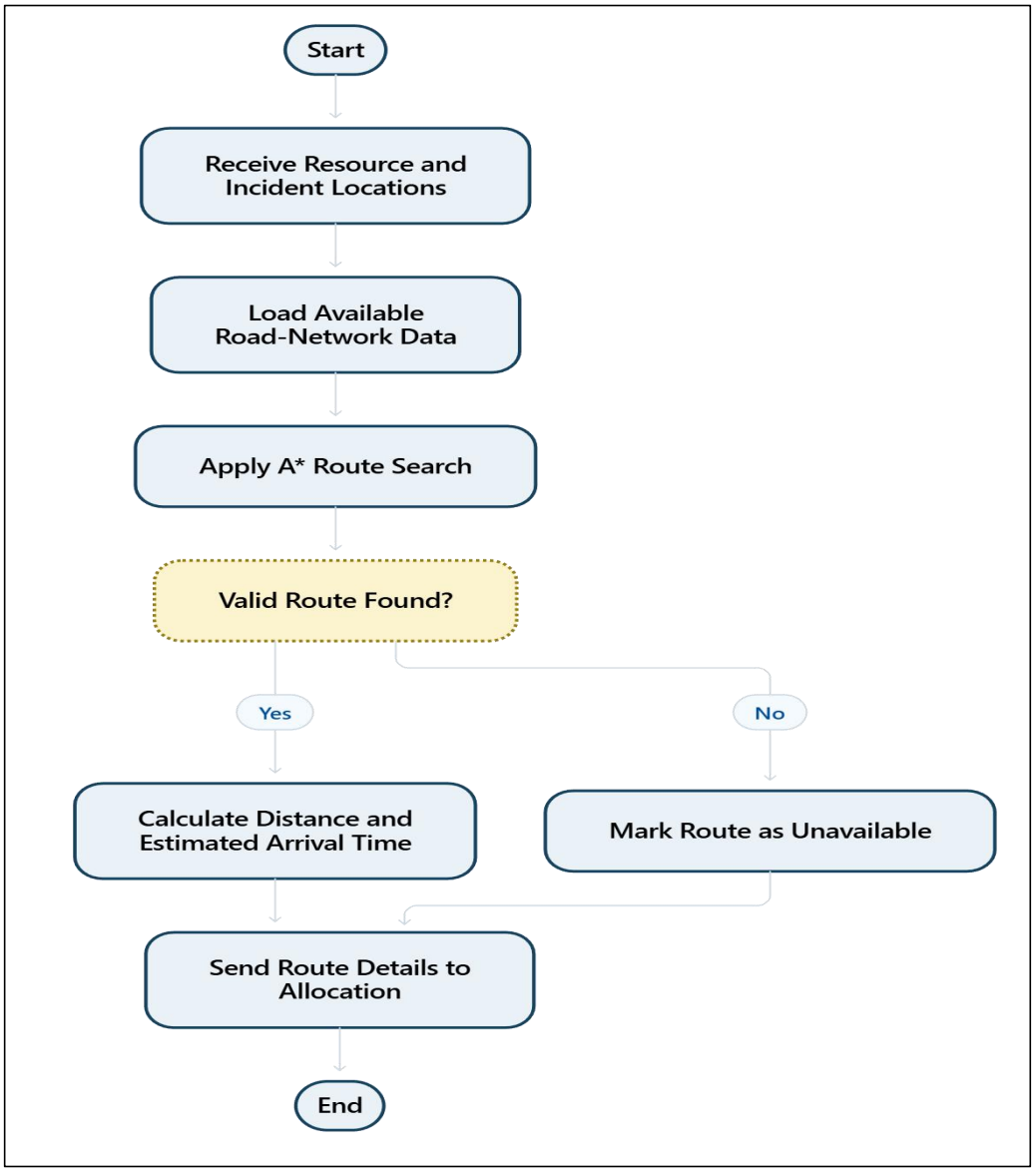


Figure 4.5 Module 05 - Route Intelligence(A*)

MODULE 06 – SCORING AND RESOURCE ALLOCATION

Purpose: Ranks feasible response options and assigns resources according to incident needs and constraints.

Implemented features

- Combine priority, suitability, travel time, and resource limits.
- Create a feasible core assignment using constraints.
- Use marginal-value scoring to improve additional assignments.
- Produce a response plan with selected resources.

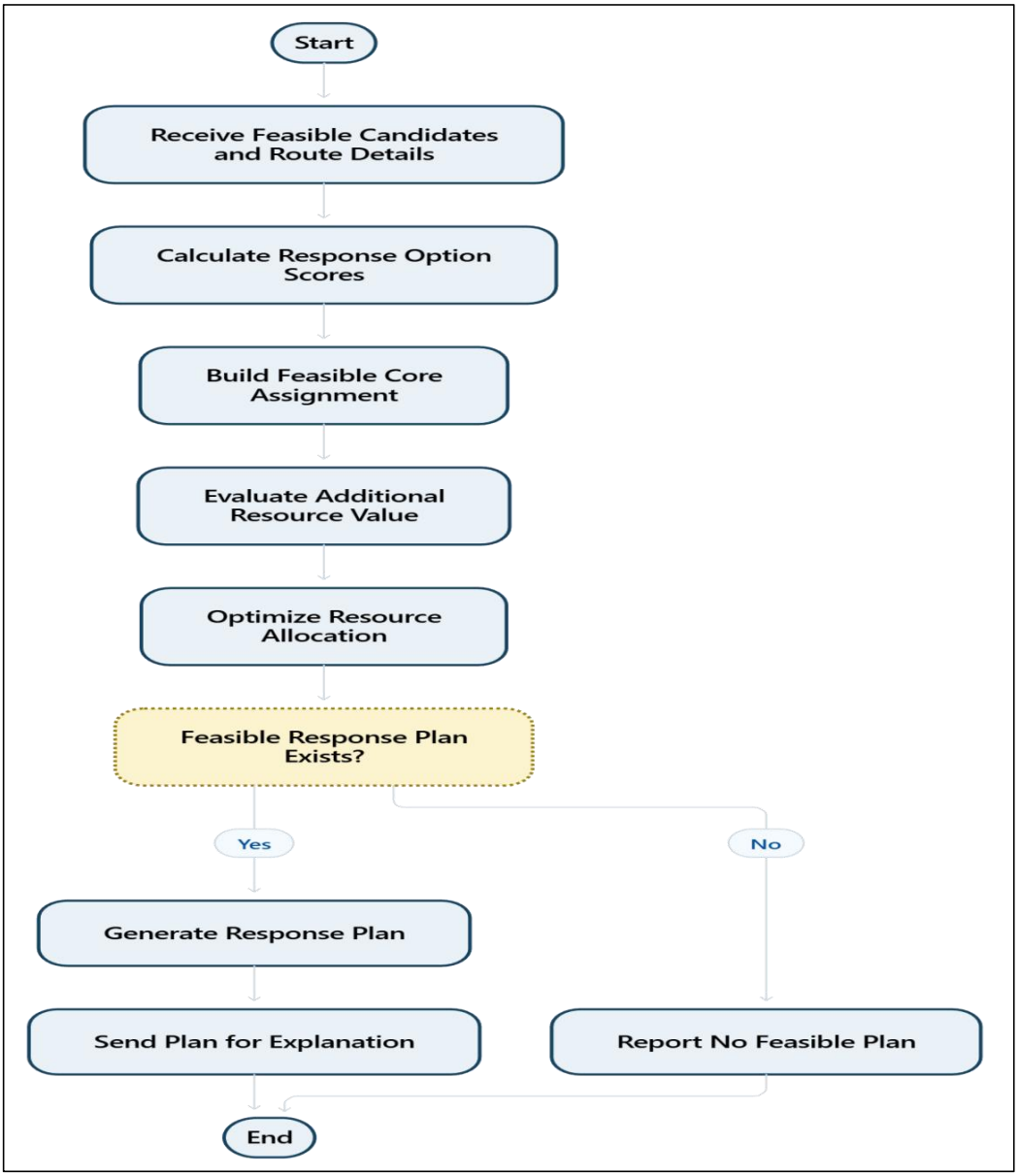


Figure 4.6 Module 06 - Scoring and Resource Allocation

MODULE 07 – EXPLANATION AND DECISION DISPLAY

Purpose: Shows the recommended response plan and explains the main reasons behind each decision.

Implemented features

- Display priority score and incident summary.
- Show selected resources, routes, and estimated arrival times.
- Explain important constraints and selection reasons.
- Clearly indicate when no feasible plan is found.

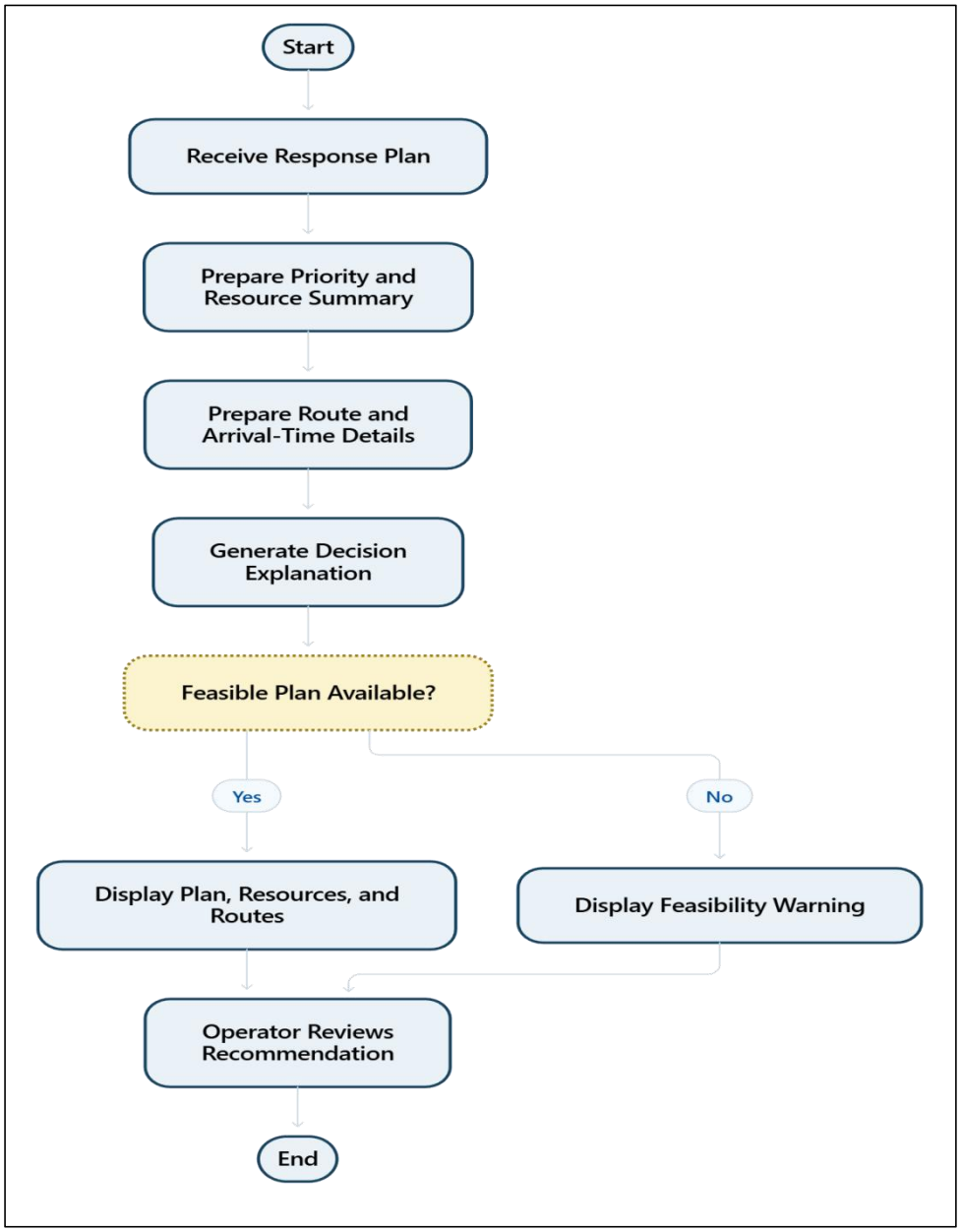


Figure 4.7 Module 07 - Explanation and Decision Planning

MODULE 08 – ADAPTATION AND RE-PLANNING

Purpose: Updates the response plan when crisis conditions or resource availability change.

Implemented features

- Accept events such as road closures, weather changes, or resource loss.
- Update the current incident and resource state.
- Re-run priority, route, constraint, and allocation steps.
- Compare the new plan with the previous plan and explain changes.

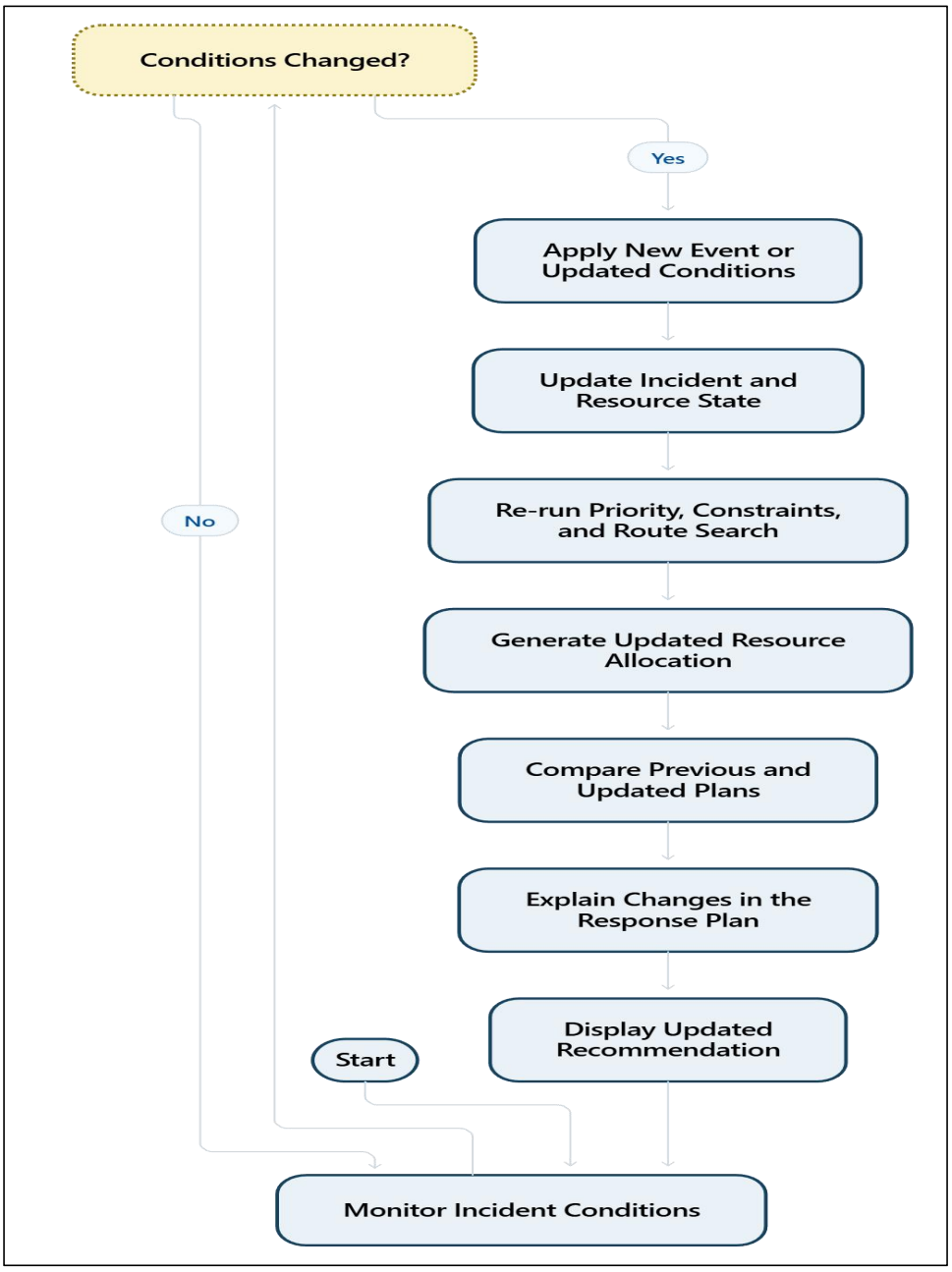


Figure 4.8 Module 08 - Adaptation and Re-planning

4.2.2 RESULTS AND DISCUSSION

The system was evaluated by an automated suite of 20 behaviour tests and by a smoke test of the interface code against stubbed Streamlit and Folium modules. They were executed with Python 3.12 on development machine without internet access, so geocoding used the offline gazetteer.

Table 4.6 Functional Test Cases and Actual Results

ID	Test	Input / condition	Expected result	Status
T01	Priority – critical flood	Flood, Critical, 2,500, heavy rain, severe damage, limited comms, difficult access; and a low case	Level CRITICAL, score ≥ 80; low-case score < 40	Pass
T02	Priority reacts to change	Fire, High, 800; then Storm and communication unavailable	Priority increases	Pass
T03	A* optimal and efficient	All ground routes of the flood plan; toy graph	A* nodes ≤ Dijkstra nodes; equal optimal cost	Pass
T04	Blocked road forces detour	Block road R01 on the boat route	Route avoids R01, status Alternative, longer ETA	Pass
T05	Inaccessible incident	Accessibility = Inaccessible	Ground units rejected (reachability); helicopter becomes primary	Pass
T06	CSP respects supply	Three variables, limited supply; and supply of two units	No group over-assigned; exactly one variable unassigned when infeasible	Pass
T07	Hard constraints reject	Storm; resource MED-1 made unavailable	Helicopter rejected (weather); MED-1 rejected (availability); no over-allocation	Pass
T08	Scores differ; boat primary	Demonstration flood	Different scores and units; rescue boat primary; all status classes present	Pass

T09	Explanations use real values	Demonstration flood	Explanation text contains actual priority, population and ETA	Pass
T10	Multiple incidents share resources	Flood + Fire + Earthquake	Ranked by priority; no group over-allocated; ambulances ≤ 3	Pass
T11	Ambulance scarcity	Three ambulances, several competing needs	Total ambulances allocated ≤ 3	Pass
T12	Adapt: road blocked	Scenario 2	Blocked road leaves the route; ETA increases; reason mentions the road	Pass
T13	Adapt: resource unavailable	Scenario 3	Primary resource changes; change recorded	Pass
T14	Adapt: weather worsens	Scenario 4	Priority and ETA increase; helicopters excluded in storm	Pass
T15	Adapt: new high-priority incident	Scenario 5	Second incident added; original response changes; no over-allocation	Pass
T16	Adapt: resource shortage	Scenario 6	Allocation changes; no over-allocation	Pass
T17	Normal operation is stable	Scenario 1	No material change; reason states no change required	Pass
T18	Restore baseline	Apply event, then restore	Original recommended action restored	Pass
T19	Why A over B	Compare two incidents	Comparison explanation produced	Pass
T20	Geocoding fallback and re-centring	Velachery; Madurai	Offline coordinates labelled simulation; network re-centred; plan produced	Pass

All 20 of 20 tests passed when executed on 14 October 2026. The tests check behaviour, for example that the decision really changes when conditions change; they are not a statistical evaluation of decision quality against real operations.

Table 4.7 gives the actual outputs for the demonstration flood incident (I001: Flood, Critical, 2,500 affected, Chennai, heavy rain, severely damaged infrastructure, limited communication, difficult access), with the primary resource and its ETA before and after each scenario.

Table 4.7 Actual Results of the Six Adaptation Scenarios for Incident I001

Scenario	Event applied	Primary resource and ETA before	Primary resource and ETA after	Plan changed
Normal operation	none	3x Rescue Boat Unit [BOAT-1], 31.9 min	3x Rescue Boat Unit [BOAT-1], 31.9 min	No
Road blocked	Road blocked: R01 (Marina Base–Egmore)	3x Rescue Boat Unit [BOAT-1], 31.9 min	3x Rescue Boat Unit [BOAT-1], 39.5 min	Yes
Resource unavailable	Resource unavailable: Rescue Boat Unit (BOAT-1)	3x Rescue Boat Unit [BOAT-1], 31.9 min	2x Rescue Boat Unit [BOAT-2], 55.1 min	Yes
Weather worsen	Weather changed: I001 -> Storm	3x Rescue Boat Unit [BOAT-1], 31.9 min	3x Rescue Boat Unit [BOAT-1], 35.1 min	Yes
New high-priority incident	New incident: Earthquake (Critical), 1,800 affected at Simulated site NW of Chennai	3x Rescue Boat Unit [BOAT-1], 31.9 min	3x Rescue Boat Unit [BOAT-1], 31.9 min	Yes
Resource shortage	Resource quantity changed: Rescue Boat Unit	3x Rescue Boat Unit [BOAT-1], 31.9 min	1x Rescue Boat Unit [BOAT-1], 31.9 min	Yes

	(BOAT-1) to 1; Resource quantity changed: Rescue Boat Unit (BOAT-2) to 1			

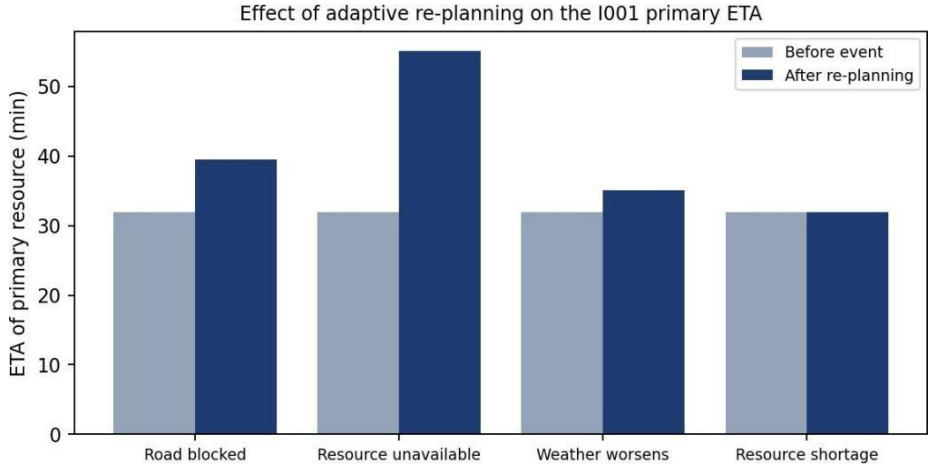


Figure 4.2 Effect of Adaptive Re-planning on the ETA of the Primary Resource

R01 (Marina Base–Egmore) became inaccessible due to a road blockage. Rescue Boat Unit for I001 had been routed via R01 (ETA 32 min). The system re-ran constraint checks and A*, and selected alternative route via the direct access road (ETA 40 min). This is the behaviour described in the project requirements: the same resource continues, the route changes and the ETA increases.

When the primary group (BOAT-1) became unavailable, the primary resource changed to the second boat group (BOAT-2) with a longer ETA (31.9 to 55.1 min). Raising the weather from heavy rain to storm increased the priority score and the ETA of the primary resource (31.9 to 35.1 min) and removed helicopters from the plan because of the weather constraint. Reducing boat quantities lowered the units allocated to the primary need from 3 to 1 for that group. In the new-incident scenario the original flood response changed because resources are now shared with the added earthquake incident.

Table 4.8 Actual Results for Three Simultaneous Incidents

Incide nt	Type	Priority	Level	Rank	Units allocated	Recommended action
I001	Flood	86.7	CRITIC A L	1	16	Deploy 3x Rescue Boat Unit from Marina Coastal Rescue Base + 8 supporting deployment(s)
I003	Earthquake	79.8	HIGH	2	17	Deploy 2x Rescue Helicopter from Tambaram Air & Rescue Base + 8 supporting deployment(s)
I002	Fire	57.9	MODER A TE	3	4	Deploy 2x Fire Engine from Ambattur Fire & Rescue Base + 2 supporting deployment(s)

The table below shows contention for limited resource types in that run. For example, the three ambulances were not allocated beyond their supply, and the single pair of helicopters went to the earthquake incident whose core search need outweighs the low-weight airlift needs of the flood.

Table 4.9 Resource Contention in the Three-Incident Run

Resource type	Supply	Potential demand	Allocation (received/requested)
Ambulance	3	11	I001: 3/4, I003: 0/5, I002: 0/2
Helicopter	2	15	I001: 0/8, I003: 2/4, I002: 0/3
Rescue Boat	5	12	I001: 5/6, I003: 0/4, I002: 0/2
USAR Team	2	9	I001: 0/3, I003: 2/5, I002: 0/1

For the ground routes of the flood plan, A* expanded 79 nodes in total against 107 for Dijkstra's algorithm, a reduction of 26%, with the same optimal travel time. The figure applies to this small simulated network and should not be generalised.

Table 4.10 Nodes Expanded by A* and Dijkstra for the I001 Ground Routes

Resource group	A* nodes	Dijkstra nodes	Travel time (min)
BOAT-1	4	5	21.9
BOAT-2	11	15	43.1
RESC-2	5	8	29.4

MED-2	5	8	29.4
MED-1	10	15	39.2
USAR-1	8	11	44.9
WATER-1	13	15	82.0
FOOD-1	13	15	82.0
RESC-1	10	15	39.2

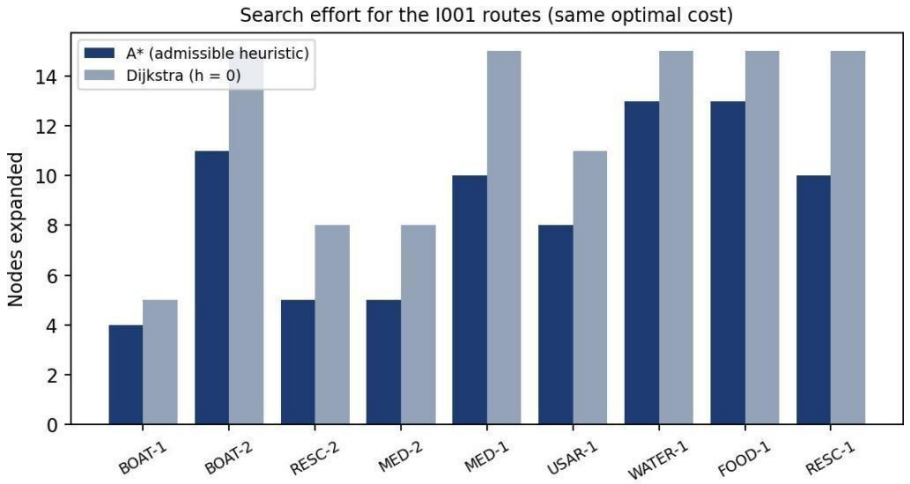


Figure 4.3 Nodes Expanded by A* and Dijkstra

The results show that the system differentiates resources (different scores and unit counts), enforces constraints (helicopter rejected in a storm, unavailable group rejected, ground resources rejected for an inaccessible incident, no group over-allocated), allocates shared resources within supply, and adapts when conditions change. In the demonstration flood, a few demands remained unmet (for example about 146 people for evacuation and 90 for airlift), which the dashboard reports instead of hiding.

All resource, road and profile data are simulated and the weights and proficiencies are assumptions, so decision quality against real operations is not established. The greedy allocation is not guaranteed to be optimal. Risk and weather penalties are simple multipliers. After a blockage the primary resource can change to a safer route observed in an earlier tuning run), so a changed primary resource does not always indicate a fault. Geocoding requires internet access. State is held in memory for one session. Authentication is absent. (viii) Performance was not benchmarked.

CHAPTER 5

CONCLUSION

This project addressed the difficulty of making prioritised, feasible and explainable response decisions when several incidents compete for limited resources and conditions keep changing. It developed Adaptive Decision Intelligence for Crisis Management, a decision-support prototype in Python and Streamlit, using classical AI techniques: a weighted priority engine, hard-constraint filtering with a backtracking CSP using MRV and forward checking, A* route search, a multi-objective heuristic, a two-phase allocation procedure and an adaptation engine that re-plans, compares the previous and current plans and explains the change. The system replaces the earlier approach of allocating one unit to every compatible resource.

The demonstrated capabilities, supported by 20 passing automated tests, are: differentiated scoring of resources; rejection of infeasible resources with reasons; route recomputation when a road is blocked; allocation of shared resources without exceeding supply across three simultaneous incidents; and plan changes in six scenarios. The contribution is an integrated, transparent pipeline that can be examined and explained, not a validated operational system.

The current limitations are the use of simulated data and assumed weights, a small road network, greedy allocation, absence of live feeds and authentication, and the pending manual validation of the browser interface. The system is a tool to support trained responders and does not replace them; its recommendations on simulated data must not be used as real emergency instructions.

Possible future work includes real-time weather, traffic and incident feeds; real road networks and a routing engine; a relational or cloud database with authentication and audit logging; optimisation methods such as integer programming for allocation; calibration of weights with expert judgement or historical data; predictive analytics; multi-agency integration; and formal usability and performance evaluation. These are future possibilities and are not implemented.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[4] A. K. Mackworth, "Consistency in networks of relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.

[5] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[6] F. Rossi, P. van Beek, and T. Walsh, Eds., *Handbook of Constraint Programming*. Amsterdam, The Netherlands: Elsevier, 2006.

[7] R. L. Keeney and H. Raiffa, *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. New York, NY, USA: Wiley, 1976.

[8] D. J. Power, *Decision Support Systems: Concepts and Resources for Managers*. Westport, CT, USA: Quorum Books, 2002.

[9] M. Turoff, M. Chumer, B. Van de Walle, and X. Yao, "The design of a dynamic emergency response management information system (DERMIS)," *J. Inf. Technol. Theory Appl.*, vol. 5, no. 4, pp. 1–35, 2004.

[10] N. Altay and W. G. Green, "OR/MS research in disaster operations management," *Eur. J. Oper. Res.*, vol. 175, no. 1, pp. 475–493, 2006.

[11] E. Galindo and R. Batta, "Review of recent developments in OR/MS research in disaster operations management," *Eur. J. Oper. Res.*, vol. 230, no. 2, pp. 201–211, 2013.

[12] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[13] United Nations General Assembly, "Transforming our world: the 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

[14] Streamlit, "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io/.

[15] A. Mukhopadhyay, G. Pettet, S. M. Vazirizade, D. Lu, A. Jaimes, S. El Said, H. Baroud, Y. Vorobeychik, M. Kochenderfer, and A. Dubey, “A review of incident prediction, resource allocation, and dispatch models for emergency management,” *Accident Analysis & Prevention*, vol. 165, Art. no. 106501, 2022, doi: 10.1016/j.aap.2021.106501.

[16] E. Rolland, R. A. Patterson, K. Ward, and B. Dodin, “Decision support for disaster management,” *Operations Management Research*, vol. 3, pp. 68–79, 2010, doi: 10.1007/s12063-010-0028-0.

[17] N. Altay and W. G. Green III, “OR/MS research in disaster operations management,” *European Journal of Operational Research*, vol. 175, no. 1, pp. 475–493, 2006, doi: 10.1016/j.ejor.2005.05.016.

[18] E. Galindo and R. Batta, “Review of recent developments in OR/MS research in disaster operations management,” *European Journal of Operational Research*, vol. 230, no. 2, pp. 201–211, 2013, doi: 10.1016/j.ejor.2013.01.039.

[19] M. Turoff, M. Chumer, B. Van de Walle, and X. Yao, “The design of a dynamic emergency response management information system (DERMIS),” *Journal of Information Technology Theory and Application*, vol. 5, no. 4, pp. 1–35, 2004.

[20] J.-H. Zhang, J. Li, and Z.-P. Liu, “Multiple-resource and multiple-depot emergency response problem considering secondary disasters,” *Expert Systems with Applications*, vol. 39, no. 12, pp. 11066–11071, 2012, doi: 10.1016/j.eswa.2012.03.016.

[21] United Nations Office for Disaster Risk Reduction, *Sendai Framework for Disaster Risk Reduction 2015–2030*. Geneva, Switzerland: United Nations, 2015.

[22] Federal Emergency Management Agency, *National Incident Management System*, 3rd ed. Washington, DC, USA: U.S. Department of Homeland Security, 2017.

[23] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, Gaithersburg, MD, USA, 2023, doi: 10.6028/NIST.AI.100-1.

[24] OpenStreetMap Contributors, “OpenStreetMap,” [Online]. Available: https://www.openstreetmap.org/. [Accessed: Oct. 22, 2026].

[25] OpenStreetMap Foundation, “Nominatim: Search engine for OpenStreetMap data,” [Online]. Available: https://nominatim.org/. [Accessed: Oct. 22, 2026].

APPENDIX

APPENDIX A — SOURCE CODE

This appendix contains the source code developed using Python and Streamlit. The code includes the user interface, input-handling logic, and the implemented components of the system. The listing below shows the project structure and the number of lines of each module; short excerpts appear in Section 4.2.1.

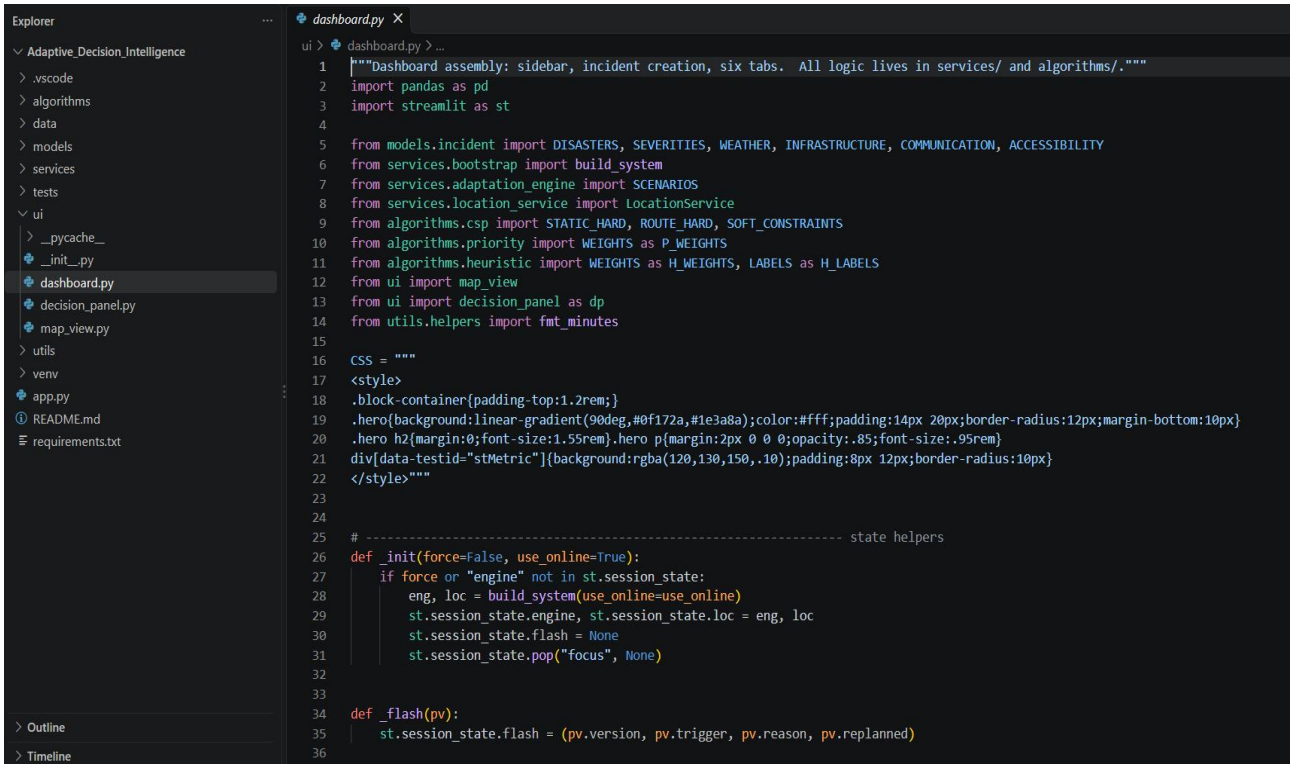


Figure A.1 Dashboard

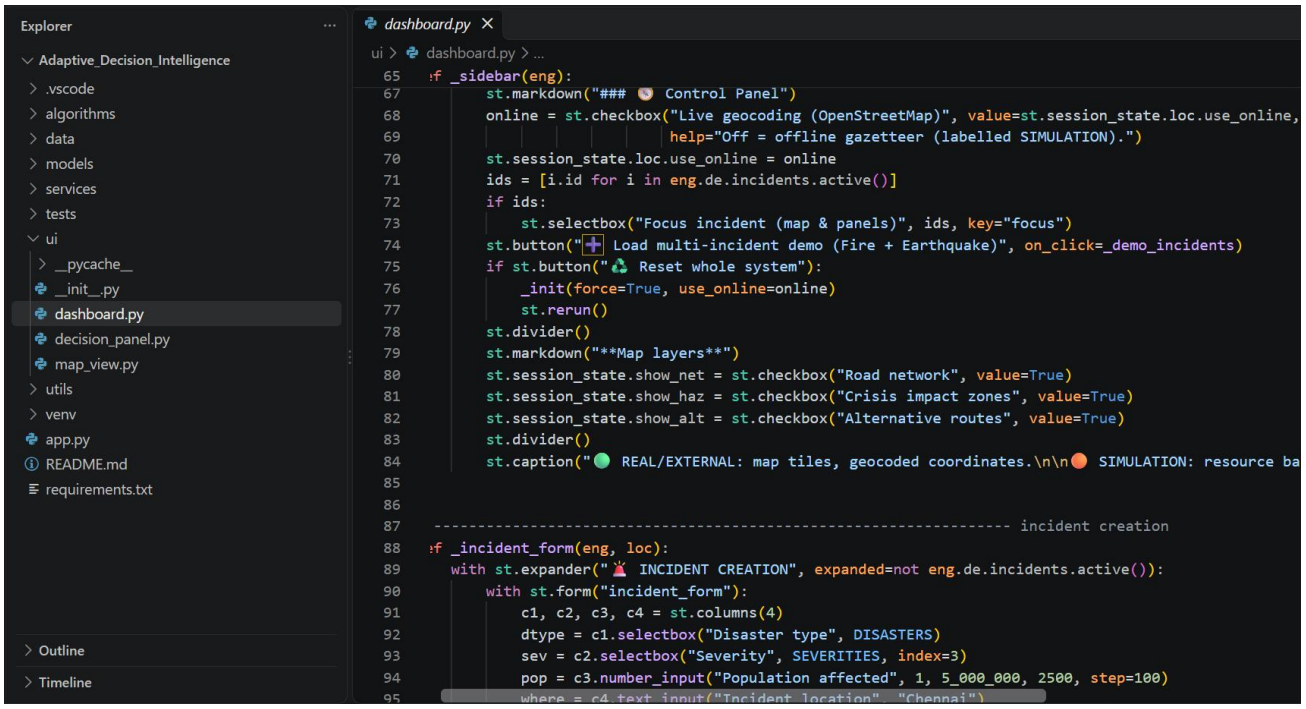


Figure A.2 Decision Panel

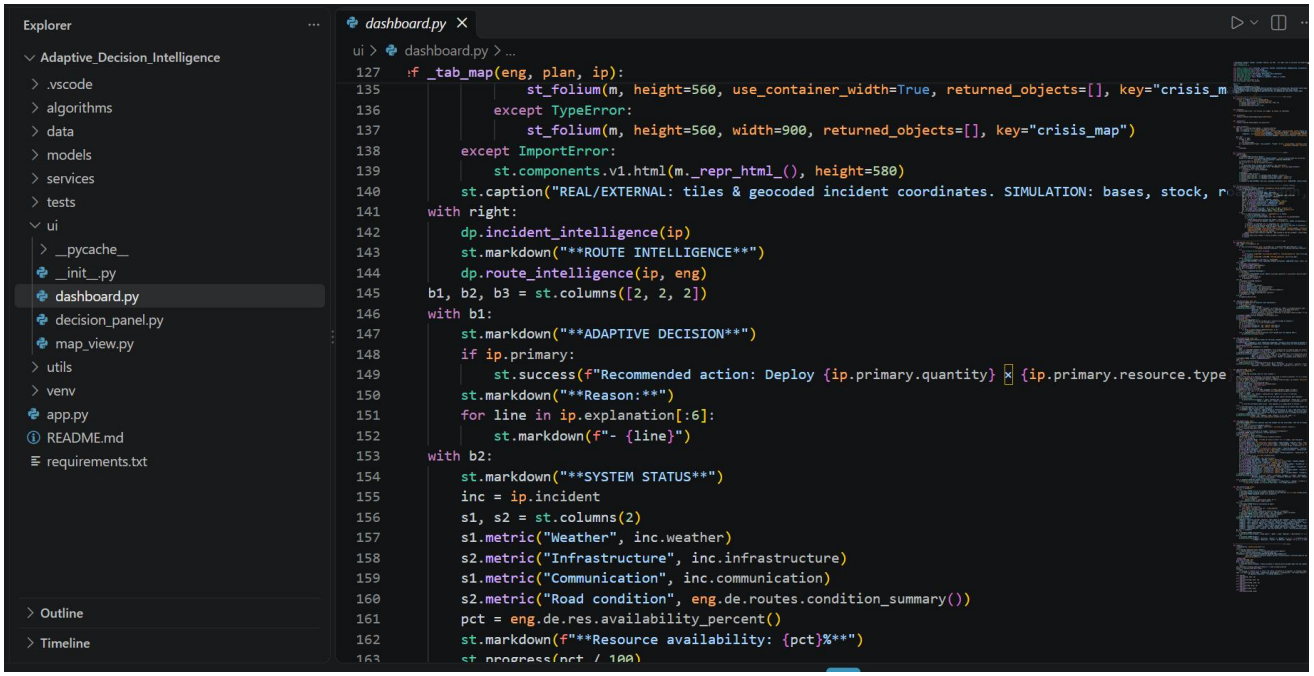


Figure A.3 App

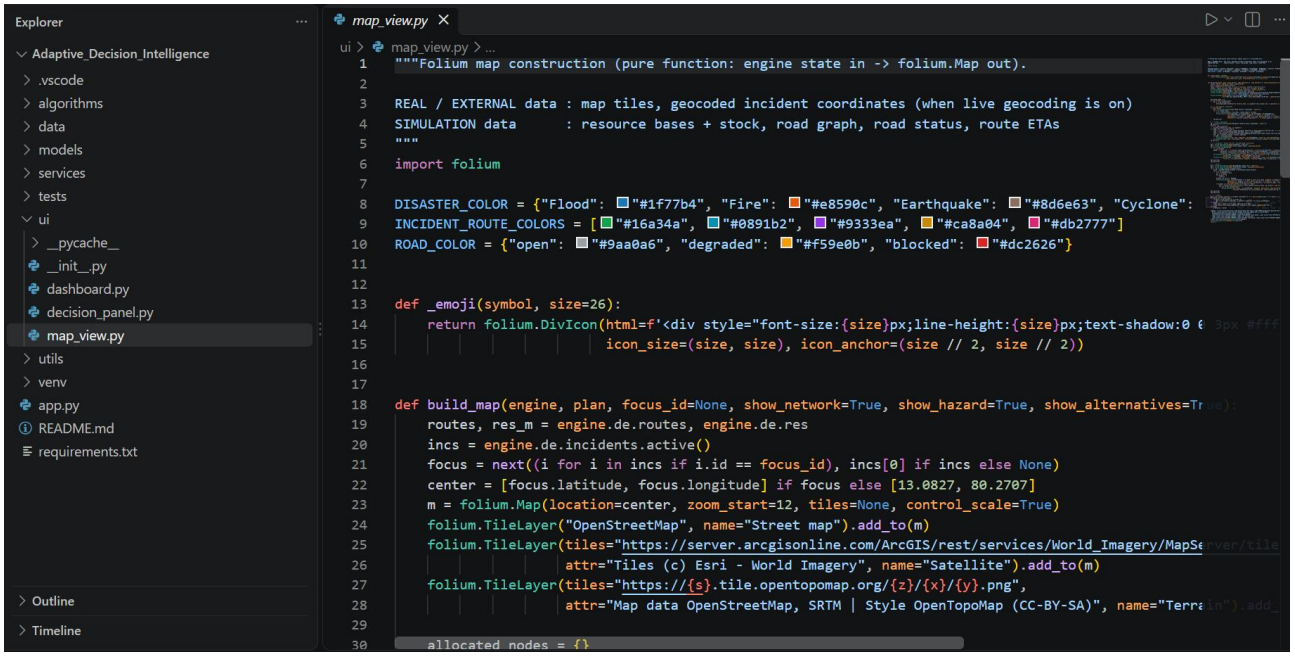


Figure A.4 Map View

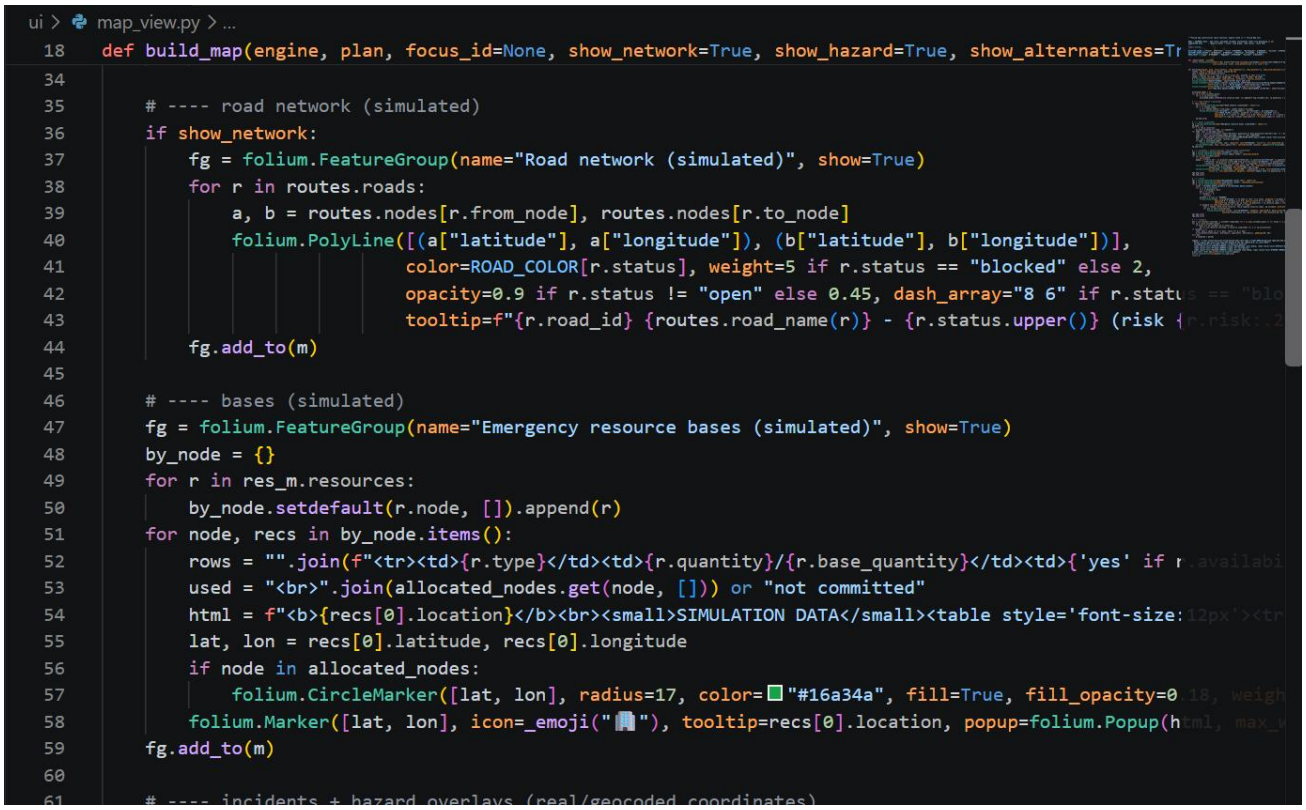


Figure A.5 Astar

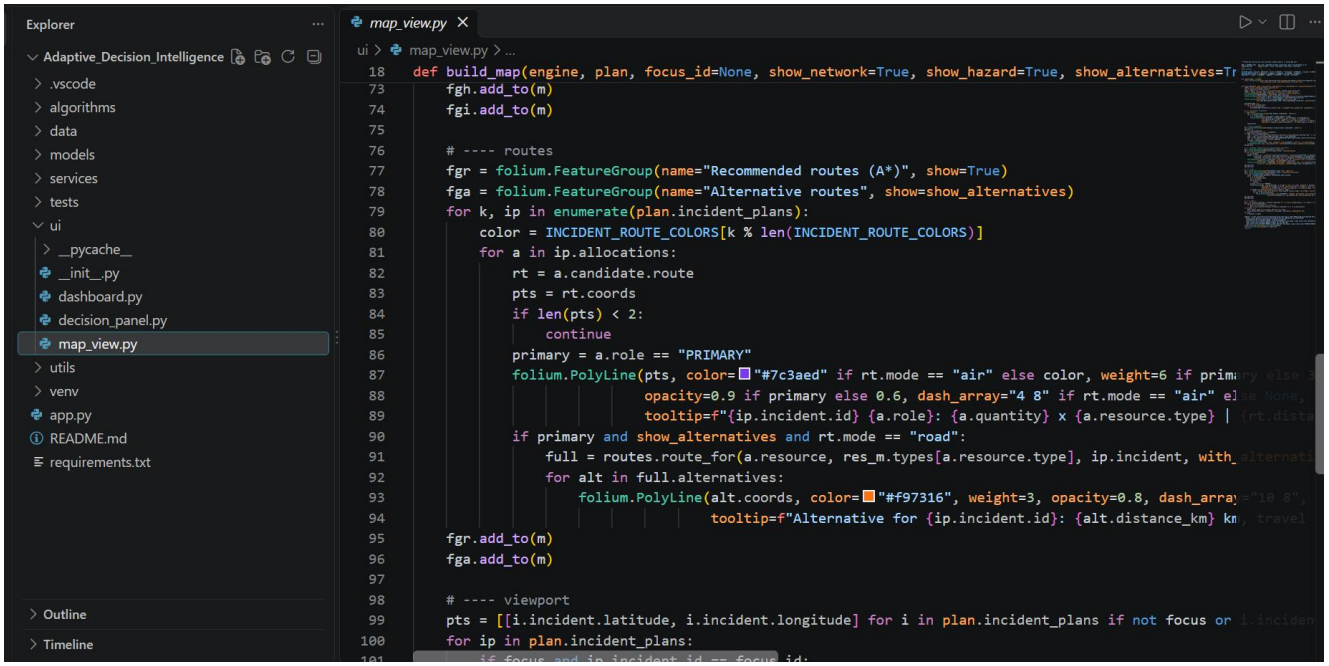


Figure A.6 Heuristic

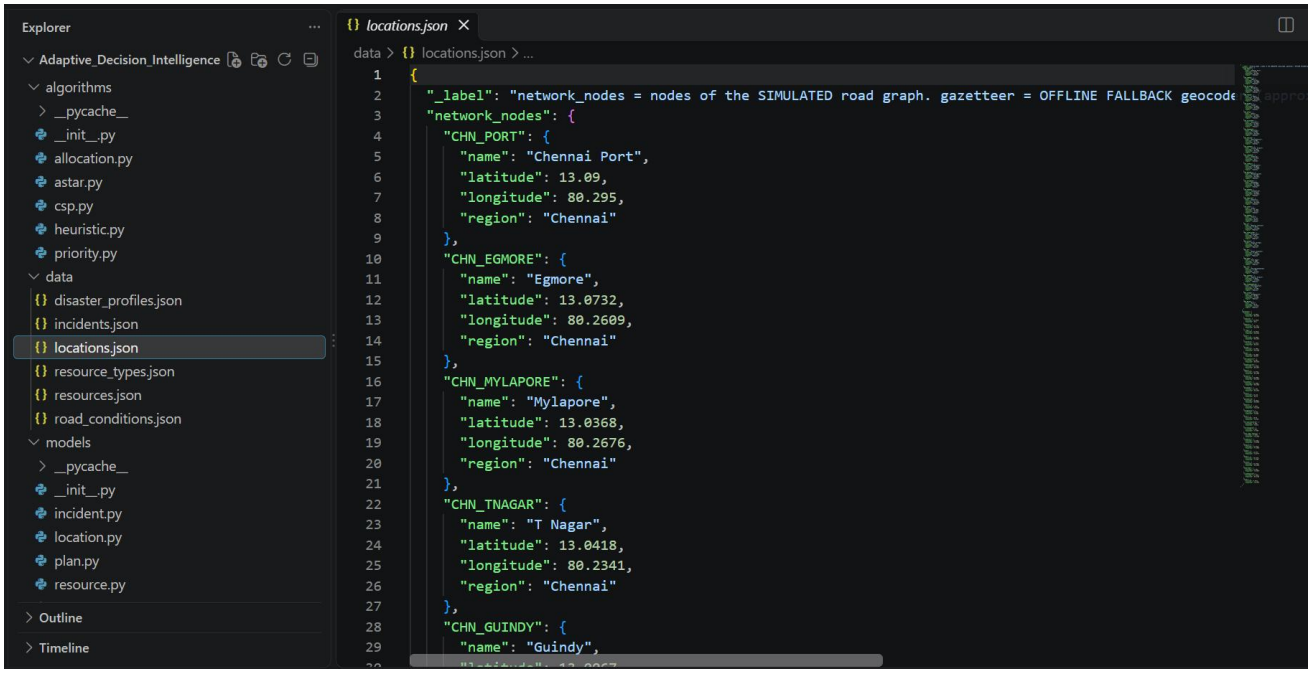


Figure A.7 Location

APPENDIX B — SAMPLE INPUTS

Table A.1 Sample Inputs for the Demonstration Incidents

Field	I001	I002	I003
Disaster type	Flood	Fire	Earthquake
Severity	Critical	High	Critical
Population affected	2,500	800	1,800
Location	Chennai	Ambattur, Chennai	Tambaram, Chennai
Weather	Heavy Rain	Light Rain	Cloudy
Infrastructure	Severely Damaged	Moderately Damaged	Severely Damaged
Communication	Limited	Normal	Limited
Accessibility	Difficult to Access	Moderate	Difficult to Access

APPENDIX C — TEST CASES

The tests and their actual results are given in Table 4.6.

Table A.2 Interface Test Cases

ID	Description	Expected result
M01	Start the application	The dashboard opens with the incident form
M02	Submit Flood / Critical / 2500 / Chennai and press ANALYZE CRISIS	Map centres on Chennai; incident marker, bases and route appear
M03	Enter an unknown location	A clear error message; no incident created
M04	Load the three-incident demo	Three markers; ranking table; contention shown
M05	Run the scenario Road blocked	Blocked road is dashed red; route and ETA change; reason displayed
M06	Run each remaining scenario and restore the baseline	Plans change as in Table 4.7; baseline restored

APPENDIX D — APPLICATION SCREENSHOTS

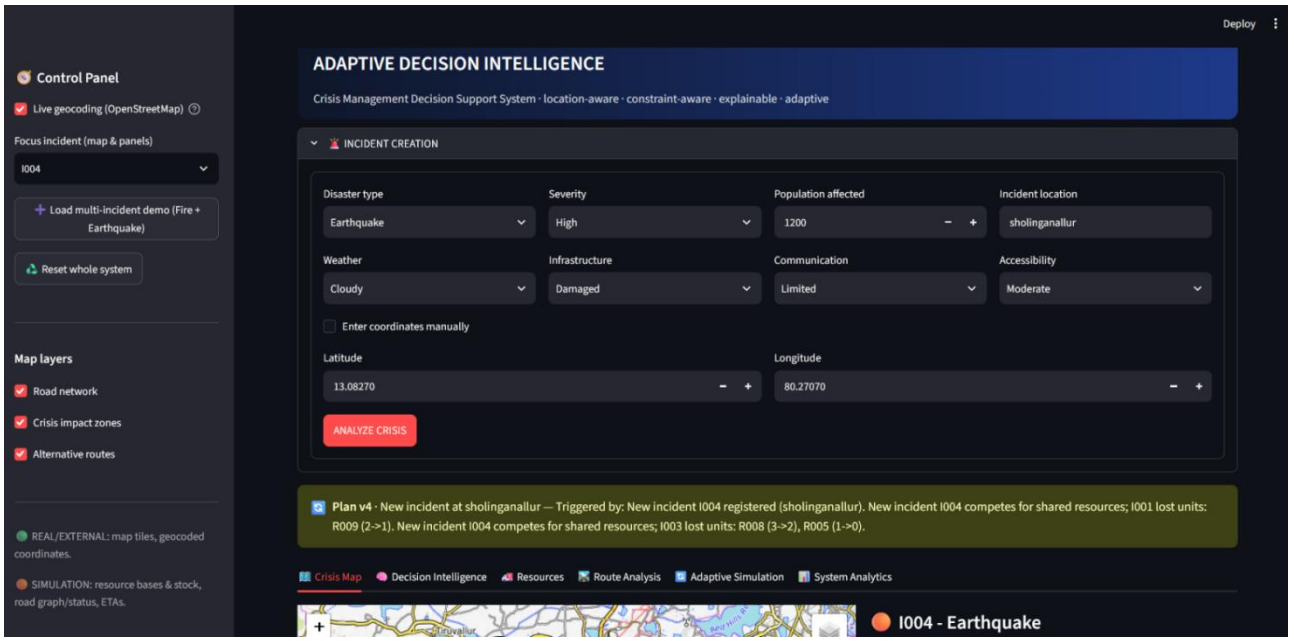


Figure D.1 Incident Creation

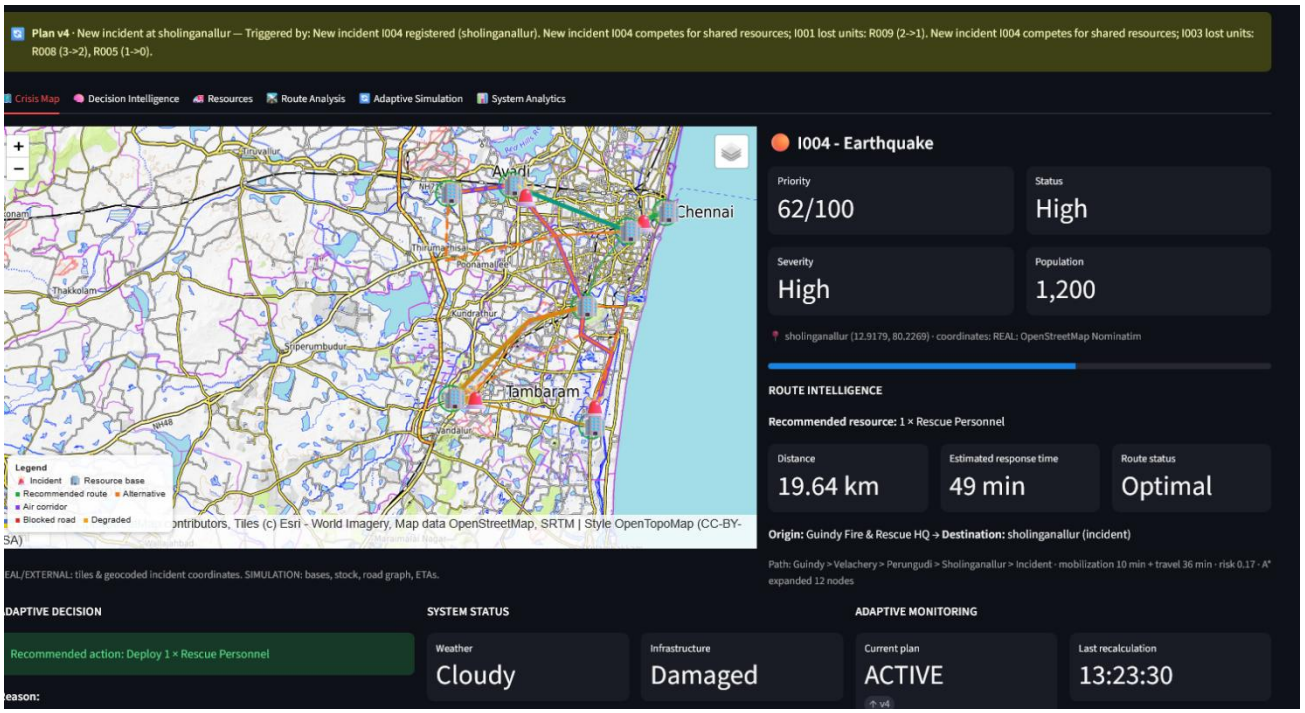


Figure D.2 Crisis Map

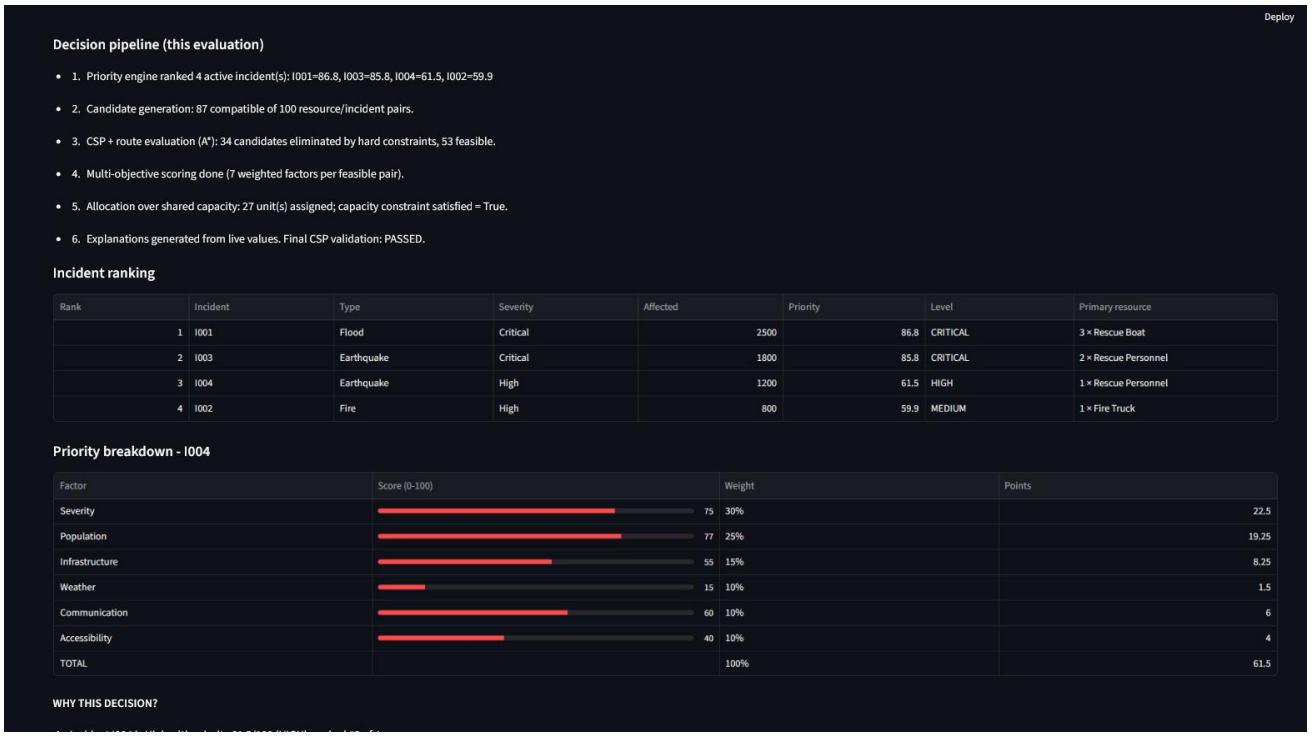


Figure D.3 Decision Intelligence

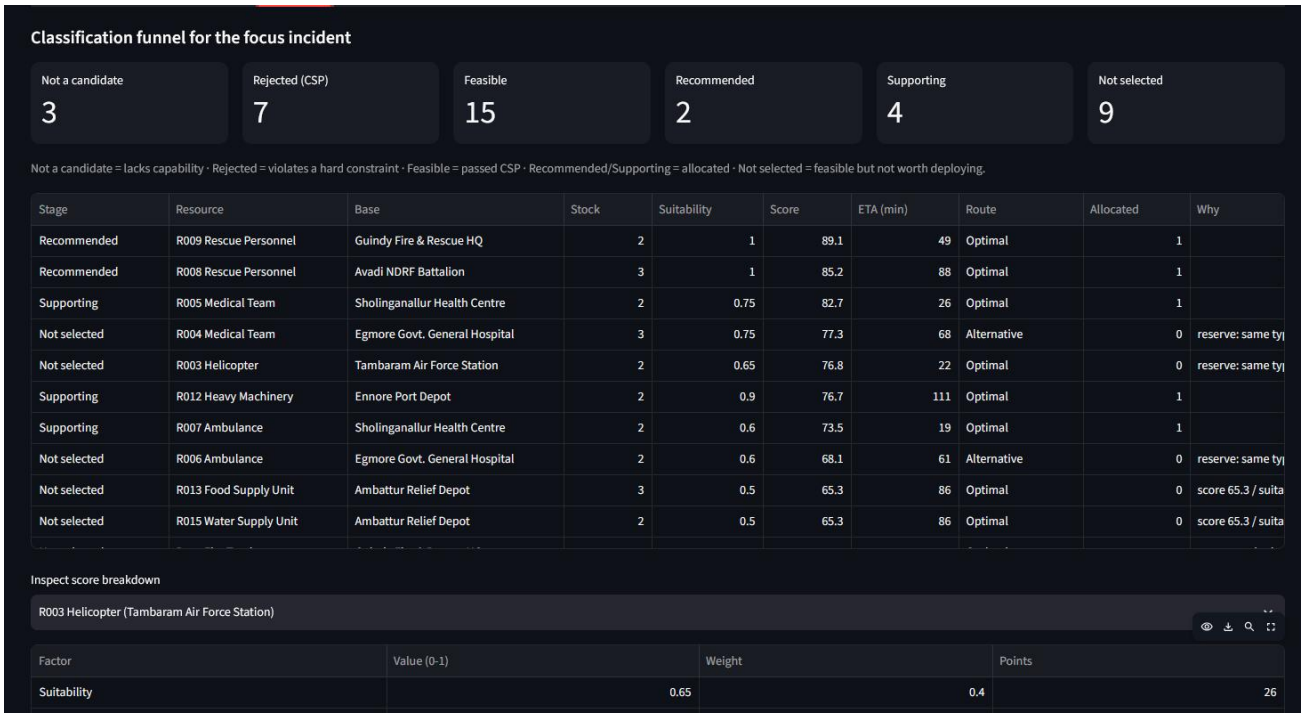


Figure D.4 Resource Allocation

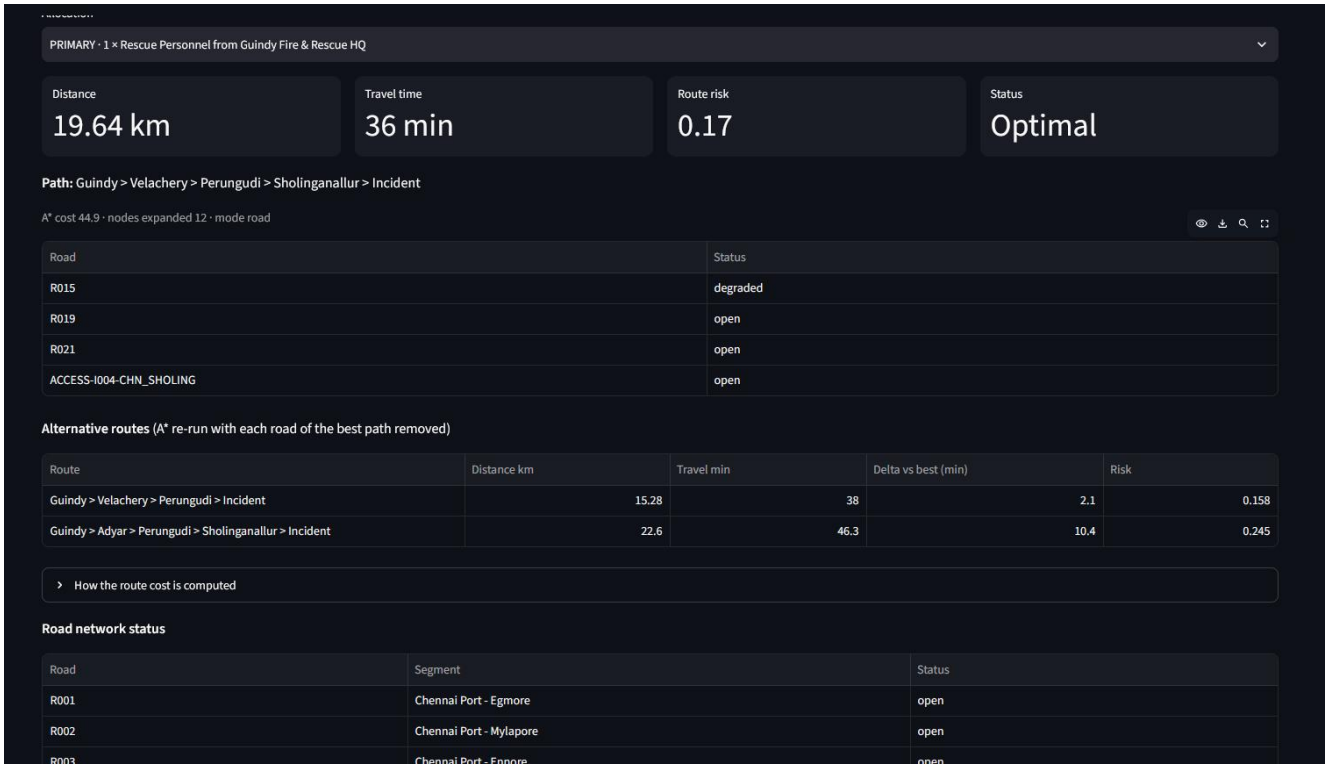


Figure D.5 Route Analysis

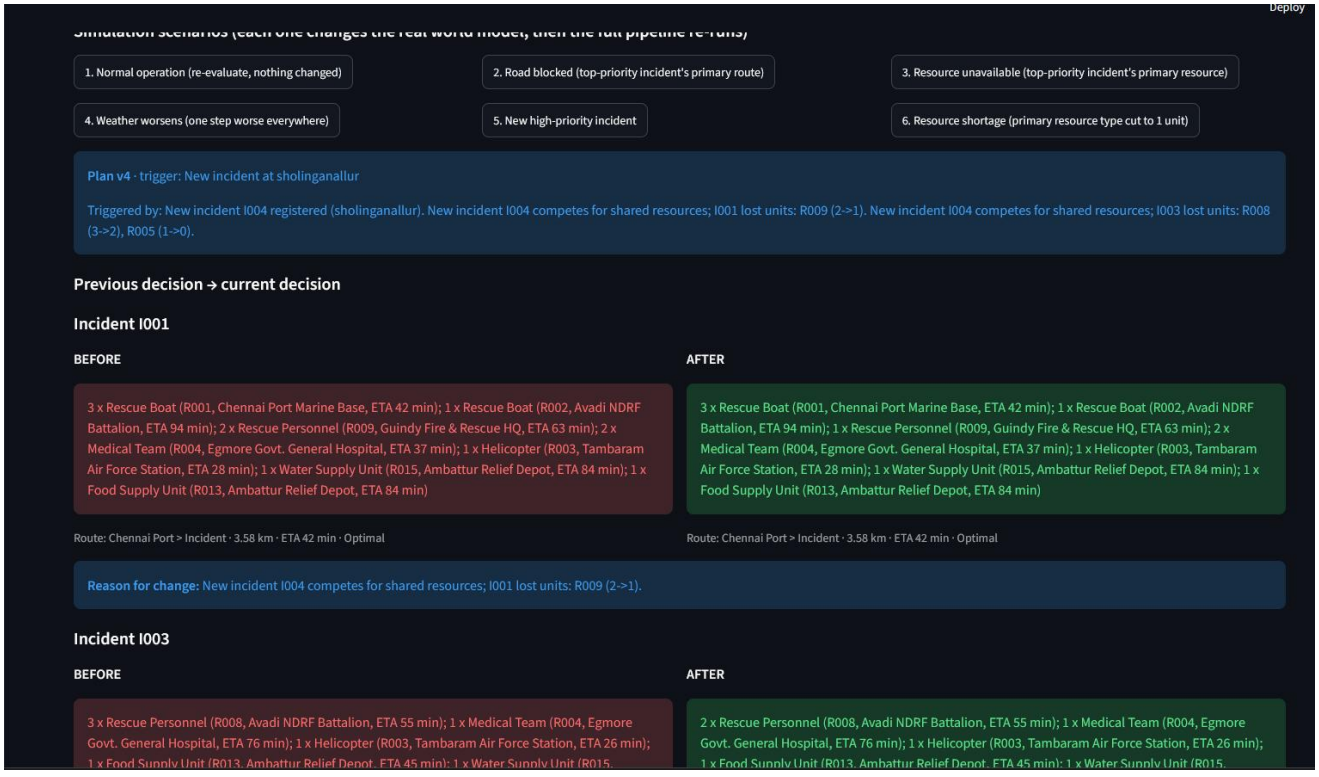


Figure D.6 Adaptive Simulation



Figure D.7 System Analysis